EEG-Based Universal Game Controller Using Brain–Computer Interface

Project Team

Jawad Saleem 22I-2423
Usman Tariq 22I-2459
Saad Abrar 22I-1238

Session 2022-2026

Supervised by

Ms. Anum Kaleem

Co-Supervised by

Mr. Akbar Ali Shah

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

September, 2025

Contents

Acknowledgements **1**

Abstract **2**

1 Introduction **3**
1.1 Background . . . 3
1.2 Existing Solutions . . . 4
1.3 Problem Statement . . . 6
1.4 Scope . . . 6
1.5 Modules . . . 7
1.5.1 Module 1: Signal Acquisition . . . 7
1.5.2 Module 2: Signal Processing and Classification . . . 7
1.5.3 Module 3: Game Integration . . . 8
1.5.4 Module 4: Evaluation and Testing . . . 8
1.6 Work Division . . . 9

2 Project Requirements **10**
2.1 Use-Case Representation . . . 10
2.2 Functional Requirements . . . 19
2.2.1 Module 1: Signal Acquisition . . . 20
2.2.2 Module 2: Signal Processing and Classification . . . 20
2.2.3 Module 3: Game Integration . . . 20
2.2.4 Module 4: Evaluation and Feedback . . . 20
2.3 Non-Functional Requirements . . . 21
2.3.1 Reliability . . . 21
2.3.2 Usability . . . 21
2.3.3 Performance . . . 21
2.3.4 Security . . . 21
2.4 Domain Model . . . 22

3 System Overview **24**
3.1 Architectural Design . . . 24
3.2 Design Models . . . 26

3.2.1 Design Models for Procedural Approach 26
3.2.1.1 Data Flow Diagrams 26
3.2.1.2 Activity Diagrams 28
3.2.1.3 State Transition Diagrams 28
3.2.1.4 Context Diagrams 30
3.2.1.5 System Sequence Diagrams 30
3.2.1.6 Package Diagram 31
3.2.1.7 Component Diagram 31
3.2.1.8 Deployment Diagram 33
3.3 Data Design 34
3.3.1 Data Structures and Processing 34
3.3.2 Data Storage and Organization 34
3.3.3 Data Flow Architecture 34
3.3.3.1 Signal Acquisition Stage 35
3.3.3.2 Preprocessing Stage 35
3.3.3.3 Feature Extraction Stage 35
3.3.3.4 Classification and Decision Making 35
3.3.3.5 Output Mapping and Game Integration 35

4 Implementation and Testing 38
4.1 Algorithm Design 38
4.1.1 Pseudocode Implementation 39
4.2 External APIs/SDKs 40
4.2.1 Key Implementation Snippets 41
4.3 Testing Details 41
4.3.1 Unit Testing 41
4.3.2 System Performance Testing 42
4.3.2.1 Saved-Model Accuracy on Available Datasets 43
4.3.2.2 Five-Fold Cross-Validation on Available Subject Datasets 43
4.3.2.3 Proxy Evaluation for Models Without Matching Archived Data . 43
4.3.3 Confusion Matrix Analysis 44
4.3.4 Latency Benchmarking 47
4.3.4.1 Window Acquisition Latency 47
4.3.4.2 Post-Window Processing Latency 47
4.3.5 Runtime Resource Utilization 47
4.3.6 Limitations and Error Analysis 48

5 Conclusions and Future Work 49
5.1 Conclusion 49
5.2 Future Work 50
5.2.1 Advanced Machine Learning Approaches 50

5.2.2 Expanded Command Vocabularies 50
5.2.3 Broader Game Integration 50
5.2.4 Hardware and Signal Processing Enhancements 51
5.2.5 User Studies and Accessibility Research 51
5.2.6 Rehabilitation and Medical Applications 52
5.2.7 Community and Infrastructure Development 52

References **54**

A Technical Diagrams and Supporting Material **55**
A.1 System Design Diagrams 55
A.1.1 Use Case Diagram 55
A.1.2 Domain Model 57
A.2 Process and Activity Diagrams 58
A.2.1 UML Activity Diagram 58
A.3 Data Flow Diagrams 60
A.3.1 System Level Data Flow 60
A.3.2 Detailed Component Data Flow 61
A.4 Signal Processing Pipeline 62
A.4.1 FBCSP Processing Pipeline 62
A.5 Testing and Validation Results 63
A.5.1 Unit Test Results 63
A.6 Implementation Resources 63
A.6.1 Software Dependencies 63
A.6.2 Hardware Specifications 64
A.6.3 Configuration Parameters 64
A.7 Troubleshooting and Common Issues 65
A.7.1 Signal Quality Issues 65
A.7.2 Classification Performance Degradation 65
A.8 Future Enhancement Opportunities 66
A.8.1 Hardware Upgrades 66
A.8.2 Software Enhancements 66
A.8.3 Accessibility and User Experience 66

List of Figures

2.1 Use Case Diagram for EEG-Based Game Control System (MindPlay) 11
2.2 Use Case Diagram for UC1 – Calibrate System 15
2.3 Use Case Diagram for UC2 – Acquire Signals 16
2.4 Use Case Diagram for UC3 – Process and Classify Signals 17
2.5 Use Case Diagram for UC4 – Control Game Actions 18
2.6 Use Case Diagram for UC5 – Provide Feedback 19
2.7 Domain Model for EEG-Based Game Control System (MindPlay) 22

3.1 High-Level System Architecture 25
3.2 Level 1 Data Flow Diagram for MindPlay showing major system components and data flows 27
3.3 Level 2 Data Flow Diagram for MindPlay showing detailed transformations within core modules 28
3.4 UML Activity Diagram of the EEG-Based Game Control Workflow including calibration, signal acquisition, feature extraction, classification, and command execution 29
3.5 Component Diagram showing system components and their dependencies 32
3.6 Deployment Diagram showing hardware-software deployment architecture 33
3.7 Level 1 Data Flow Diagram showing system-level data flows and major components 36
3.8 Level 2 Data Flow Diagram detailing data transformations within each processing module 37

4.1 Flowchart of the FBCSP Signal Processing Pipeline 39
4.2 Python Code Snippet for LSL Data Acquisition 41
4.3 Unit Testing Results for Blink Detection Module 42
4.4 Saved-Model Accuracy Summary for Primary and Proxy Evaluations 45
4.5 Selected Confusion Matrices for Evaluated Models 46

A.1 Use Case Diagram for MindPlay - Showing interactions between users, system, and game environment 55
A.2 Domain Model for MindPlay - Core entities and relationships 57
A.3 UML Activity Diagram of the MindPlay EEG-Based Game Control Workflow showing calibration, signal acquisition, preprocessing, feature extraction, classification, and command execution 59

A.4 Level 1 Data Flow Diagram for MindPlay - Overview of major data flows and system components 60
A.5 Level 2 Data Flow Diagram for MindPlay - Detailed flows within core processing modules 61
A.6 Filter Bank Common Spatial Pattern (FBCSP) Signal Processing Pipeline - Showing frequency decomposition and spatial filtering stages 62
A.7 Blink Detection Unit Test Results - Validation of eye blink detection algorithm performance 63

List of Tables

1.1 Comparison of Existing EEG-Based Gaming Solutions 5

2.1 Detailed Use Case UC1 – Calibrate System 12
2.2 Detailed Use Case UC2 – Acquire Signals 12
2.3 Detailed Use Case: UC3 – Process and Classify Signals 13
2.4 Detailed Use Case: UC4 – Control Game Actions 13
2.5 Detailed Use Case: UC5 – Provide Feedback 14

4.1 Saved FBCSP+LDA Model Accuracy on Matching Available Datasets 43
4.2 Five-Fold Cross-Validation Accuracy on Available Subject Epoch Files 43
4.3 Proxy Best-Available Evaluation for Models Without Matching Subject Data 44
4.4 Confusion Matrix for Subject S21 Saved Model - Binary Classification (Rest vs. Motor Imagery) 44

A.1 Critical System Configuration Parameters 65

Acknowledgements

We would like to express our sincere gratitude to Ms. Anum and Mr. Akbar Ali Shah for their exceptional guidance, technical expertise, and continuous support throughout the development of the MindPlay project. Their insights into brain-computer interface design and signal processing have been invaluable in shaping our research and implementation approach.

We extend our thanks to the National University of Computer and Emerging Sciences (FAST-NUCES) for providing the necessary resources, laboratory facilities, and computational infrastructure required for signal processing, machine learning model development, and system integration.

We are grateful to all the participants who volunteered for our user trials and testing phases. Their feedback and cooperation were crucial in validating the system's usability and performance in real-world scenarios.

Finally, we appreciate the open-source communities behind PyLSL, scikit-learn, and other libraries that enabled rapid prototyping and development of our BCI system. Their contributions have made advanced signal processing and machine learning accessible to researchers worldwide.

Abstract

MindPlay: EEG-Based Universal Game Controller Using Brain-Computer Interface

Brain-Computer Interfaces (BCIs) represent a transformative frontier in human-computer interaction, enabling direct communication between the brain and external devices. This project presents MindPlay, an open-source, real-time EEG-based gaming application that translates motor imagery and eye blink signals into game control commands. Our system integrates a non-invasive 3-electrode motor imagery montage (Cz, C3, C4) positioned over the central motor cortex with advanced signal processing techniques to achieve responsive, accessible gaming control for users with motor impairments.

The system architecture combines the Lab Streaming Layer (LSL) framework for multi-modal data synchronization with the Filter Bank Common Spatial Pattern (FBCSP) algorithm coupled with Linear Discriminant Analysis (LDA) for real-time classification of motor imagery vs. rest states. Our implementation achieves classification accuracy of 70.0% on training data with 5-fold cross-validation accuracy of 61.9% ($\pm$5.4%). Integration with the Tuxemon game environment demonstrates practical viability for real-world gaming scenarios.

This thesis documents the complete lifecycle of MindPlay development, including requirements analysis, system design, signal processing pipeline implementation, and comprehensive evaluation through unit and system-level testing. The work addresses critical gaps in existing BCI gaming platforms by providing an accessible, reproducible, and extensible open-source solution that combines academic rigor with practical usability.

Key contributions include: (1) a validated FBCSP+LDA pipeline optimized for reduced electrode configurations, (2) seamless hardware-game integration through keyboard emulation, (3) open-source methodology enabling reproducibility and community contributions.

Keywords: Brain-Computer Interface, EEG, Signal Processing, Motor Imagery, Game Control, Real-Time Classification, Accessibility, Filter Bank Common Spatial Pattern, Linear Discriminant Analysis

Chapter 1

Introduction

1.1 Background

This document specifies the requirements and design for the **EEG-Based Universal Game Controller Using Brain–Computer Interface**, a brain-computer interface (BCI) application that enables users to control game actions using neural signals and motion data. The system represents a significant advancement in human-computer interaction (HCI) research by making gaming more accessible to individuals with motor impairments and introducing novel immersive control paradigms.

Brain-Computer Interfaces have emerged as transformative technologies that directly translate neural signals into machine commands, bypassing traditional muscular control pathways [1]. Over the past two decades, EEG (electroencephalography) has become the most practical non-invasive method for capturing brain activity due to its low cost, portability, and ease of use compared to fMRI or invasive intracranial recordings [2]. In recent years, gaming has been identified as a compelling application domain for BCI research, as it combines real-time interaction demands, immediate feedback mechanisms, and user engagement metrics that enable rapid iteration and validation of BCI systems [3].

Traditional gaming interfaces—keyboards, mice, and handheld controllers—have remained largely unchanged for decades. While effective for the general population, these devices create accessibility barriers for approximately 1 billion people worldwide living with disabilities that affect motor control. Furthermore, the entertainment and accessibility communities recognize that BCI-driven gaming can offer not only functional access but also novelty and immersion, opening new avenues for inclusive digital experiences.

The motivation for this project stems from the convergence of three factors: (1) the maturation of consumer-grade EEG headsets with improved signal quality, (2) the availability of robust open-source machine learning frameworks for real-time classification, and (3) the critical need for accessible gaming technologies that serve underrepresented populations. By developing MindPlay,

we aim to demonstrate that reliable, real-time EEG-based game control is achievable within a constrained development timeline and with limited resources.

1.2 Existing Solutions

Several commercial and research-based BCI systems for gaming currently exist in the market. A critical analysis of these systems reveals significant gaps that our project seeks to address. The following table compares leading EEG-based gaming solutions:

Table 1.1: Comparison of Existing EEG-Based Gaming Solutions

System Name	System Overview	System Limitations
Emotiv Cortex	Commercial EEG headset (14 electrodes) with proprietary API; supports mental commands, facial expressions, and motion detection. Offers SDKs for game integration.	Proprietary platform lock-in; expensive hardware ($800+); limited customization; requires commercial licensing; SDK limitations for research-grade control.
Muse Headband	Consumer-grade EEG device (4 electrodes, dry) focusing on meditation and attention metrics. Limited command set; primarily cloud-based.	Insufficient electrode coverage for robust motor imagery detection; minimal game integration support; lag in real-time performance; limited open-source community.
OpenBCI Cyton	Open-source hardware platform (8-16 electrodes) with modular design. Supports custom signal processing via Python/MATLAB.	Steep learning curve for non-experts; requires significant calibration; lacks out-of-box gaming integration; limited pre-built classification models.
Smarting EEG	Medical-grade EEG headset configured for a 3-electrode motor imagery montage (Cz, C3, C4) with low-latency LSL streaming.	Higher cost ($15,000+); specialized hardware; requires technical expertise; limited pre-built game interfaces.
MindPlay (This Project)	Parallel EEG and gyroscope control streams, real-time FBCSP+LDA classification, seamless Tuxemon integration, open methodology.	Addresses above limitations through open-source design, accessible hardware setup, and practical gaming validation.

Key Gaps Addressed by MindPlay:

- **Proprietary Lock-in:** Unlike Emotiv and Muse, MindPlay uses open-source frameworks (PyLSL, scikit-learn) and provides transparent, reproducible methodology.

- **Accessibility:** Combines affordable hardware (Smarting) with custom calibration protocols that reduce training time.
- **Multimodal Control:** Uses parallel EEG and gyroscope command streams, improving control precision and user experience beyond EEG-only systems.

1.3 Problem Statement

Traditional methods of interacting with computer games rely heavily on physical input devices such as keyboards, mice, and handheld controllers. Although these devices are effective, they create accessibility barriers for individuals with motor impairments and limit the immersive potential of gaming experiences. Current EEG-based gaming solutions, such as those provided by commercial vendors like Emotiv and Muse, are largely confined to proprietary platforms with restricted customization options. Moreover, existing research prototypes suffer from issues such as lengthy calibration times, poor generalization between users, limited command sets, and inadequate real-world usability for seamless gaming integration.

As a result, players with disabilities and researchers exploring BCI applications remain unable to use brain signals to interact with diverse games in real-time without significant financial investment or technical expertise. This system is being developed to overcome these challenges by creating an interface that translates EEG-based mental states and gyroscope data into standard gaming inputs using open-source methodologies. In doing so, the software addresses accessibility needs, provides a novel immersive input method for gamers, and offers researchers a flexible tool for exploring BCI applications beyond specialized test environments. The proposed solution addresses directly the issues of ecosystem lock-in, limited command sets, and inadequate real-world usability, ultimately enabling a reliable brain-controlled interface for modern gaming that is both affordable and reproducible.

1.4 Scope

The scope of this project is to design and implement a brain-computer interface (BCI) system that allows users to control basic actions in the game Tuxemon using only EEG signals and gyroscope input. The system is intentionally limited to two distinct commands which will be mapped to two in-game actions. This keeps the project within a manageable range while still demonstrating the practicality of EEG-based control in gaming. In addition to EEG, gyroscope data will be incorporated to capture simple head movements, thereby enhancing control without adding excessive complexity.

The solution will focus on real-time acquisition of brain signals through Smarting EEG devices, preprocessing of the data to remove noise, and classification of EEG patterns into meaningful

control commands. Integration with the game Tuxemon will be achieved by mapping EEG and gyro signals to predefined keyboard inputs so that users can interact with the game environment seamlessly. The proposed project does not aim to cover a wide range of EEG commands or test across multiple games; instead, it establishes a proof-of-concept that EEG-driven gaming is both feasible and enjoyable. The boundaries of the system exclude medical-grade diagnostics, multi-class EEG recognition beyond two commands, or integration with complex commercial gaming platforms.

The project is also limited to single-user interaction and does not involve multi-player EEG synchronization. Its focus is strictly on demonstrating functionality and usability within a controlled experimental setup. By narrowing the scope to only two commands and one test game, the project ensures achievable goals, measurable outcomes, and a clear path to evaluation. Ultimately, the system serves as a foundation for future extensions where additional commands, more sophisticated models, and broader game support can be introduced.

1.5 Modules

The proposed project is divided into four modules, each addressing a distinct part of the system workflow. These modules ensure the project is structured from signal capture to final gameplay evaluation, covering acquisition, processing, integration, and performance testing.

1.5.1 Module 1: Signal Acquisition

This module establishes a stable connection with the Smarting EEG headset and gyroscope to capture real-time data streams. It handles the initialization of the Lab Streaming Layer (LSL) framework, manages device connections, and synchronizes multiple data channels. The module is responsible for ensuring continuous, reliable data flow to downstream processing components.

1. Acquire EEG signals using a 3-electrode motor imagery montage (Cz, C3, C4) and head movement data (3-axis gyroscope) in real-time.
2. Synchronize EEG and gyroscope channels for unified processing using LSL timestamps.

1.5.2 Module 2: Signal Processing and Classification

This module transforms raw EEG and gyro data into usable control commands through filtering, feature extraction, and machine learning classification. It applies domain-specific signal processing techniques to handle noise, artifacts, and inter-subject variability. The module is the computational core of the BCI pipeline.

1. Apply noise filtering, artifact removal, and bandpass filtering (4–30 Hz frequency range).
2. Extract features using Filter Bank Common Spatial Patterns (FBCSP) and compute log-variance features [4].
3. Classify two mental commands (e.g., rest vs. motor imagery) using Linear Discriminant Analysis (LDA) [5].
4. Interpret gyroscope data for head movement detection and integration with EEG commands.

1.5.3 Module 3: Game Integration

This module connects the classified EEG and gyro outputs to the Tuxemon game by mapping them to keyboard actions. It emulates user input at the operating system level, ensuring seamless interaction with the game environment. The module abstracts away low-level hardware details from the game.

1. Map EEG-based commands to keypress actions (e.g., arrow keys for direction, spacebar for selection).
2. Use gyro data for camera rotation or additional movement parameters [6].
3. Implement command validation and debouncing to prevent accidental triggers.

1.5.4 Module 4: Evaluation and Testing

This module assesses the system's performance, usability, and reliability through comprehensive testing protocols. It generates quantitative metrics, user feedback, and performance visualizations for iterative system improvement.

1. Measure EEG classification accuracy, false positive rate, and system response time.
2. Conduct user trials to evaluate accessibility, immersion, and user satisfaction.
3. Generate reports, visualizations, and statistical analyses of performance metrics.
4. Document latency, calibration time, and cross-subject generalization performance.

1.6 Work Division

Role	Responsibilities
Jawad Saleem (22I-2423)	Primary responsibility for Module 1 (Signal Acquisition) and hardware integration. Manages EEG headset setup, LSL stream configuration, and real-time data buffering. Conducts unit testing for data acquisition pipeline and ensures synchronization between EEG and gyroscope channels.
Usman Tariq (22I-2459)	Primary responsibility for Module 2 (Signal Processing and Classification). Implements FBCSP algorithm, LDA classifier training, and feature extraction. Develops real-time classification engine and optimizes computational performance for low-latency operation. Conducts hyperparameter tuning and model validation.
Saad Abrar (22I-1238)	Primary responsibility for Module 3 (Game Integration) and Module 4 (Evaluation & Testing). Develops game interface, implements keyboard emulation, and designs user testing protocols. Conducts system-level testing, generates performance reports, and manages project documentation.
Ms. Anum (Supervisor)	Provides technical guidance on BCI signal processing, machine learning approaches, and experimental design. Reviews requirements, design decisions, and testing methodologies. Ensures alignment with research standards and academic integrity.
Mr. Akbar Ali Shah (Co-supervisor)	Offers technical support on implementation details, software architecture, and deployment strategies. Assists with troubleshooting and optimization. Provides feedback on code quality and system reliability.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-Case Representation

Since the proposed system is an interactive, user-driven application, the **Use Case** technique is most appropriate for describing its behavior. The use case diagram illustrates the interaction between the user and the main components of the system.

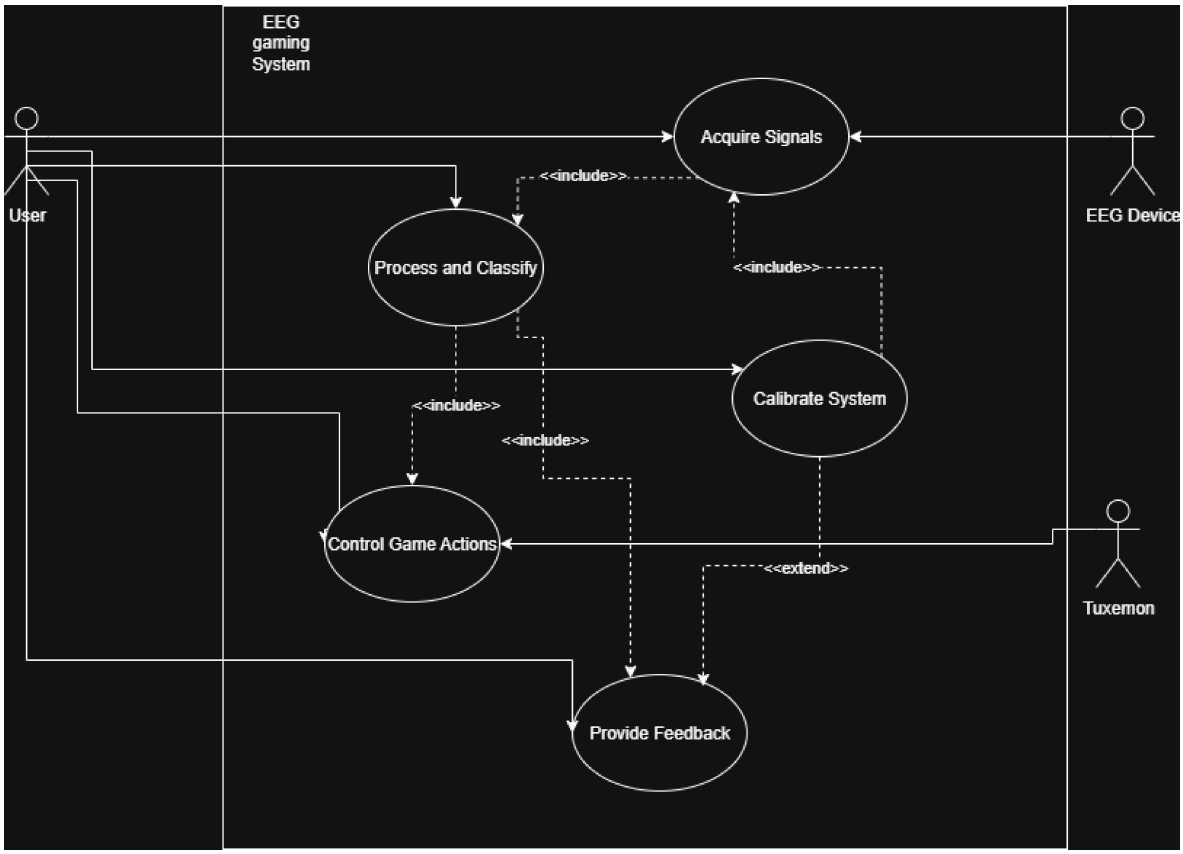


Figure 2.1: Use Case Diagram for EEG-Based Game Control System (MindPlay)

Primary Use Cases

- **UC1: Calibrate System —** The user performs short guided tasks (rest and motor imagery) to train the binary FBCSP+LDA classifier. Blink detection is calibrated separately using a threshold-based detector.
- **UC2: Acquire Signals —** EEG and gyroscope data are streamed in real time from the headset.
- **UC3: Process and Classify Signals —** The system filters, preprocesses, and classifies neural activity into motor imagery vs. rest states via FBCSP+LDA. Blink events are detected independently via amplitude thresholding.
- **UC4: Control Game Actions —** Classified motor imagery commands are mapped to in-game actions such as move or attack. Blink is mapped separately as a select/confirm command.
- **UC5: Provide Feedback —** The system gives real-time feedback (visual/audio) on detected commands or signal quality.

Detailed Use Cases

Table 2.1: Detailed Use Case UC1 – Calibrate System

Use Case:	UC1 – Calibrate System
Actors	User (Gamer or Person with Disability)
Precondition	EEG headset is connected and recognized by the system.
Postcondition	System collects sufficient labeled EEG and gyro data for personalized calibration.
Main Flow	1. The system prompts the user to begin calibration. 2. The user performs guided tasks: “Rest” (passive baseline) and “Motor Imagery” (left hand movement). 3. EEG and gyroscope data are recorded and labeled for each task. 4. The FBCSP+LDA classifier is trained on the binary (Rest vs. Motor Imagery) labeled data. 5. In a separate step, blink detection threshold is calibrated using intentional eye blink samples. 6. A performance summary is displayed after completion.
Alternate Flow	If EEG signal quality drops below a threshold, the system pauses calibration and asks the user to adjust the headset.
Exceptions	Calibration timeout if data is not collected within 15 minutes.

Table 2.2: Detailed Use Case UC2 – Acquire Signals

Use Case:	UC2 – Acquire Signals
Actors	BCI System
Precondition	Headset is connected and calibration completed.
Postcondition	Continuous EEG and gyroscope streams are available for processing.
Main Flow	1. The system initializes data acquisition from the EEG headset. 2. Data streams are synchronized with timestamps. 3. The system buffers real-time data in sliding windows of 4 seconds with 0.5-second step overlap. 4. Buffered data are passed to the preprocessing module.
Alternate Flow	If connection drops, the system attempts auto-reconnection.
Exceptions	Sensor disconnection or signal noise beyond acceptable limits triggers a warning message.

Table 2.3: Detailed Use Case: UC3 – Process and Classify Signals

Use Case:	UC3 – Process and Classify Signals
Actors	BCI System
Precondition	Real-time EEG and gyro data are being received.
Postcondition	Classified commands (Motor Imagery vs. Rest) and independent blink events are generated for game mapping.
Main Flow	1. The system filters EEG signals (4–30 Hz) to remove noise and artifacts. 2. Eye blinks are detected using amplitude thresholding (peak-to-peak > 80 µV) as a separate detector, independent of the FBCSP+LDA classifier. 3. The FBCSP+LDA pipeline processes filtered data and produces binary class scores (Rest vs. Motor Imagery). 4. Gyroscope data are processed to calculate head orientation or tilt. 5. The system maps EEG classification and blink/gyro information to independent control streams.
Alternate Flow	If classifier confidence is below threshold, the system maintains the previous state (“Rest”).
Exceptions	Missing EEG or gyro data for more than 2 seconds pauses classification until restored.

Table 2.4: Detailed Use Case: UC4 – Control Game Actions

Use Case:	UC4 – Control Game Actions
Actors	User, Tuxemon
Precondition	Classifier outputs control commands in real time.
Postcondition	Game responds to EEG/gyro-based input commands.
Main Flow	1. Classified outputs (Motor Imagery or Rest) are mapped to predefined game controls. 2. “Motor Imagery” (typically left hand imagery) triggers movement or action in the game. 3. “Blink” (detected via separate threshold detector) acts as a selector/confirm command. 4. Gyroscope tilt controls direction or camera movement. 5. Commands are sent as keyboard/mouse events to the game.
Alternate Flow	If classification confidence drops below threshold, the system holds input until stable.
Exceptions	If the game is not responding, the system queues commands and retries once connection resumes.

Table 2.5: Detailed Use Case: UC5 – Provide Feedback

Use Case:	UC5 – Provide Feedback
Actors	User
Precondition	System is actively processing EEG/gyro data.
Postcondition	User receives visual and/or audio feedback on performance and connection quality.
Main Flow	1. The system continuously monitors signal quality and classifier accuracy. 2. Feedback is displayed via visual indicators (bars, icons, or probability values). 3. The user can adjust headset placement if poor signal quality is detected. 4. At session end, a performance summary (accuracy, latency, false positives) is shown.
Alternate Flow	In minimal-UI mode, feedback is provided as audio tones or LED indicators.
Exceptions	If headset is disconnected, system alerts the user and pauses feedback until reconnection.

Individual Use Case Diagrams

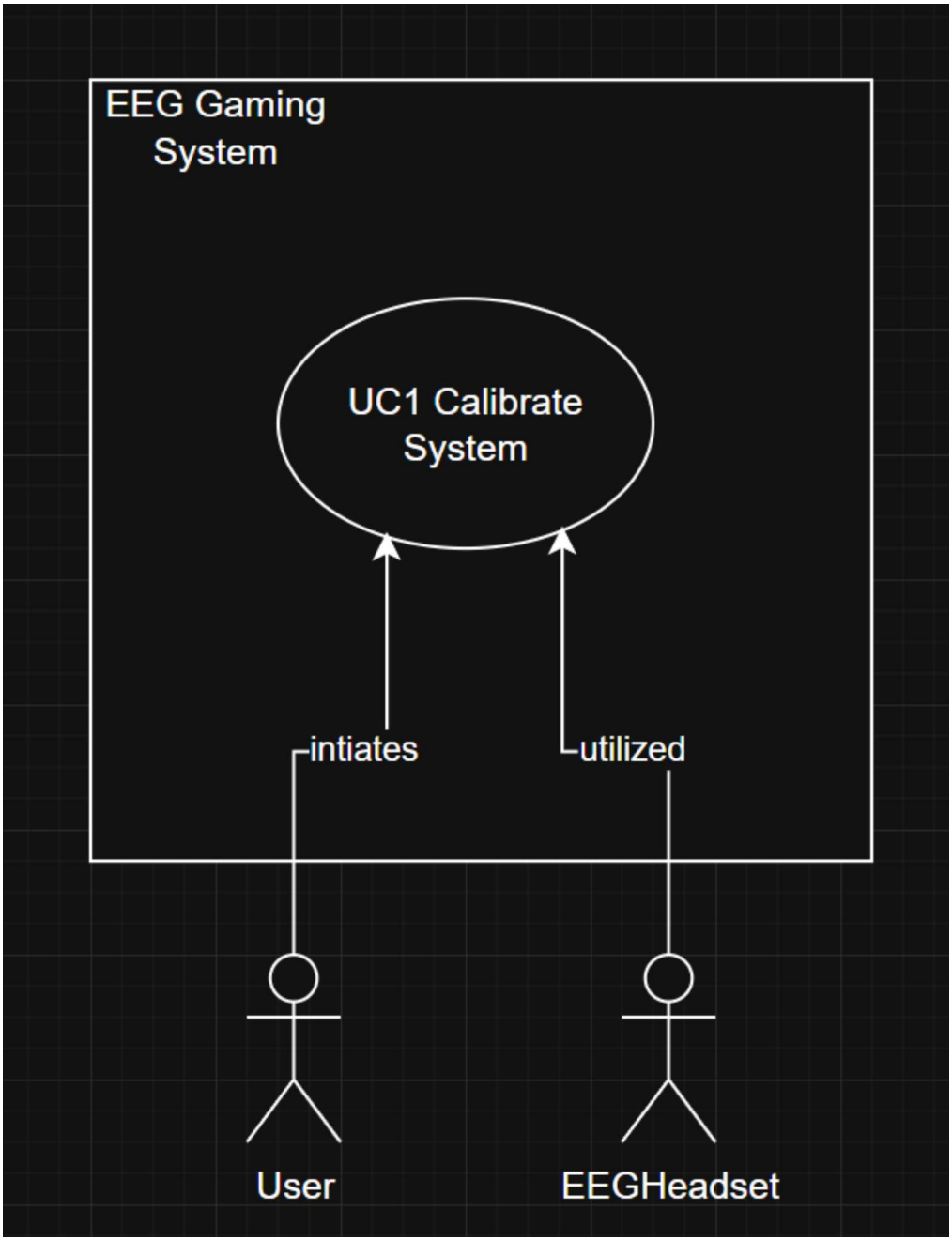


Figure 2.2: Use Case Diagram for UC1 – Calibrate System

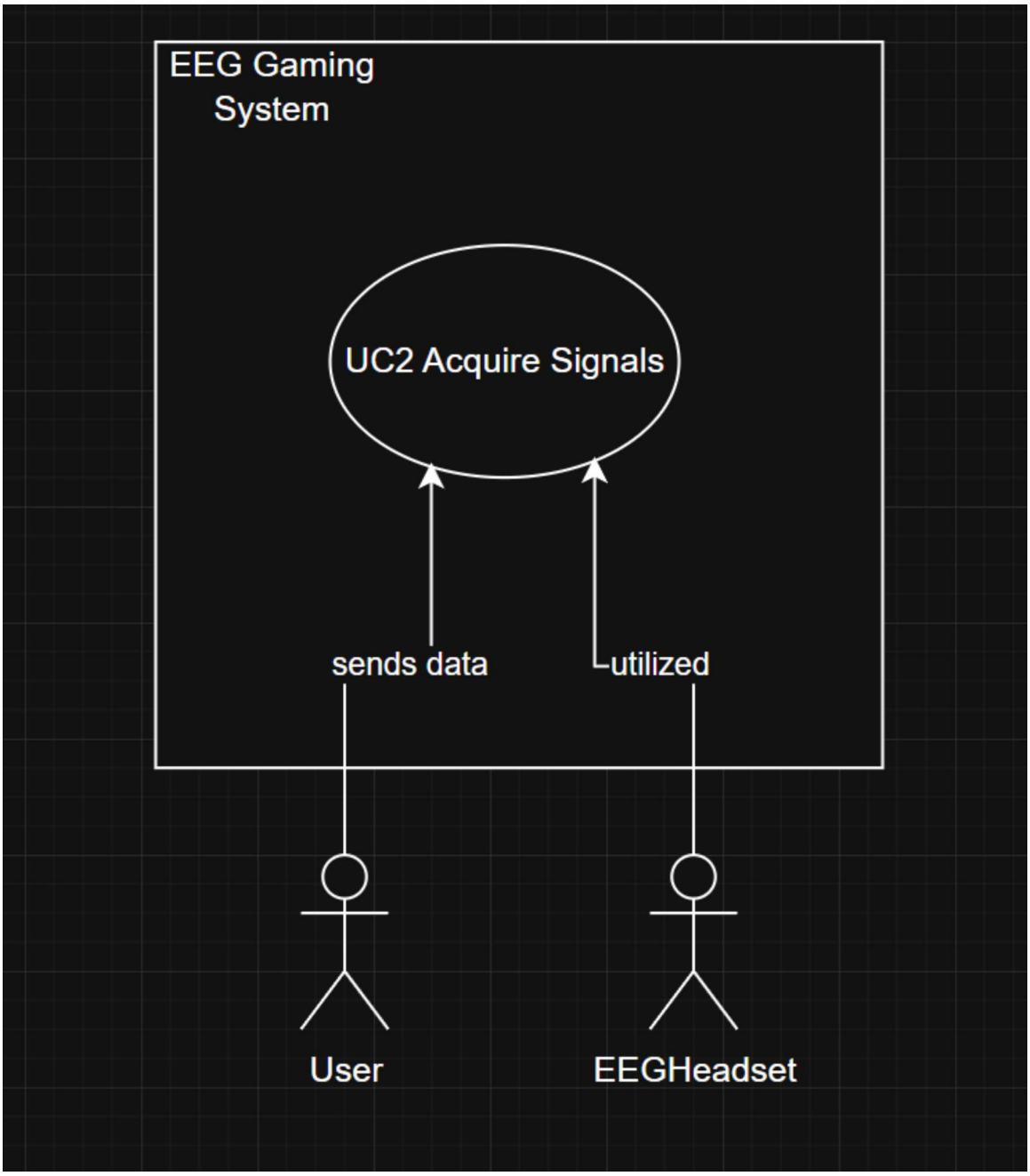


Figure 2.3: Use Case Diagram for UC2 – Acquire Signals

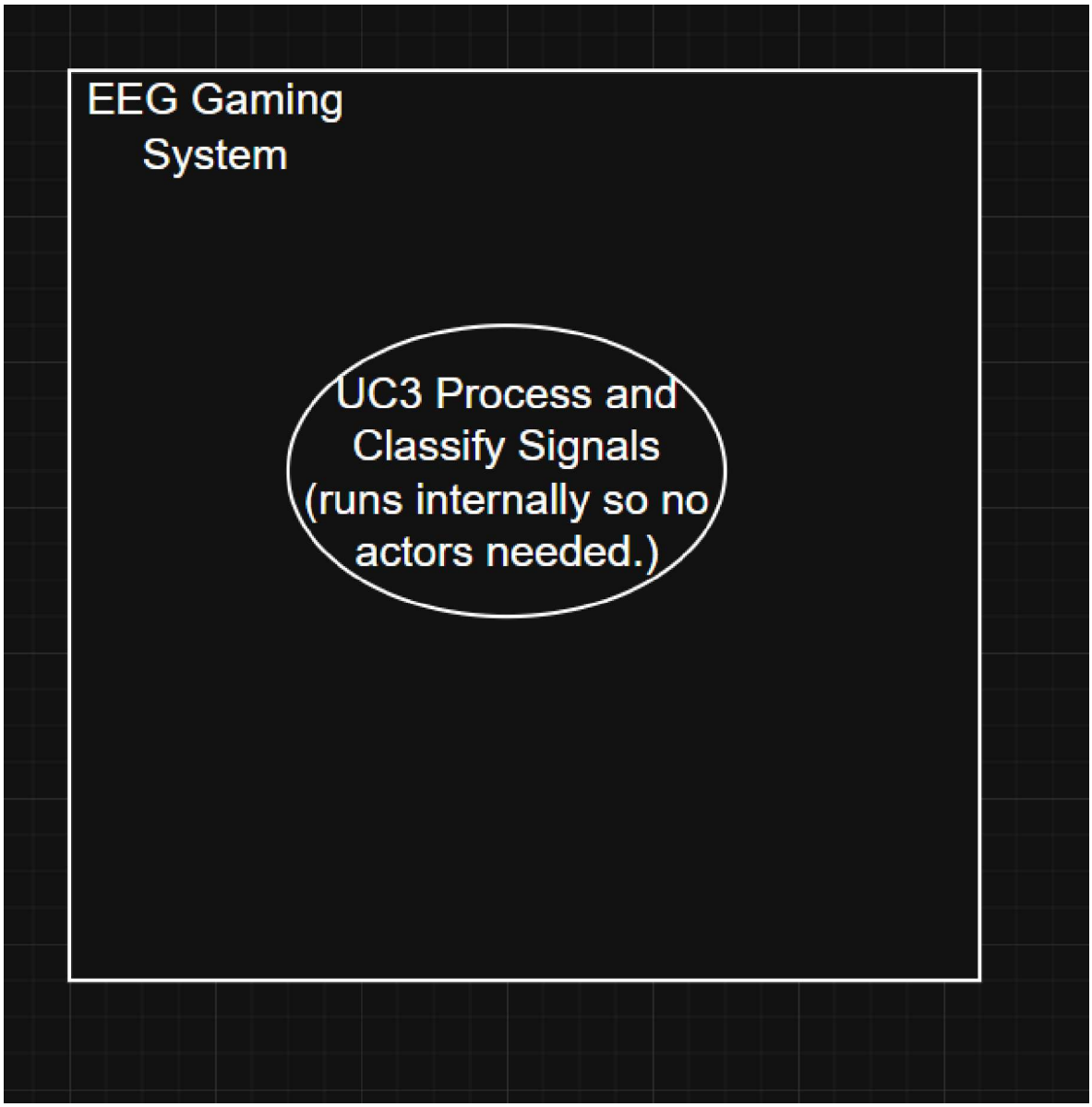


Figure 2.4: Use Case Diagram for UC3 – Process and Classify Signals

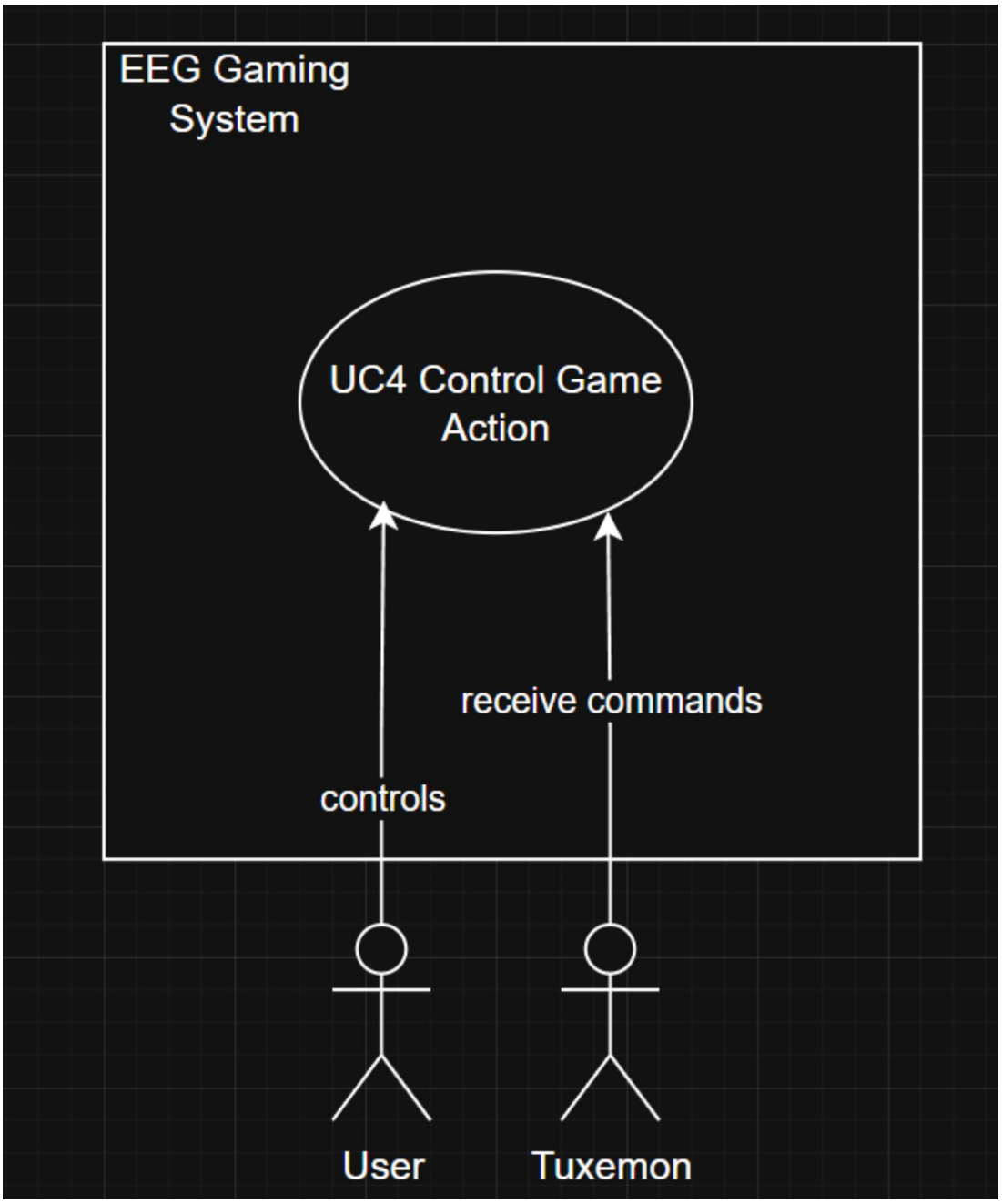


Figure 2.5: Use Case Diagram for UC4 – Control Game Actions

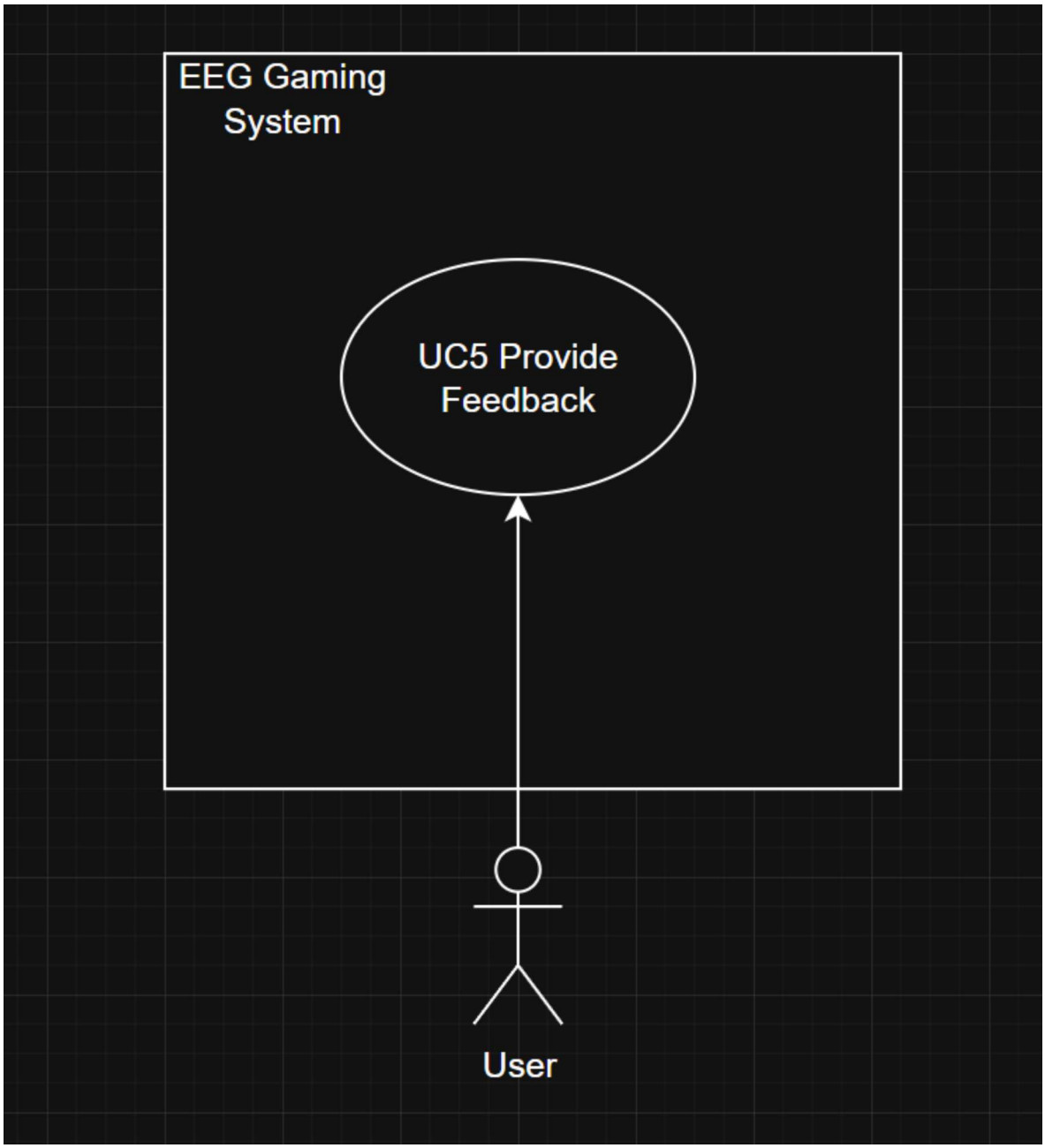


Figure 2.6: Use Case Diagram for UC5 – Provide Feedback

2.2 Functional Requirements

The functional requirements are categorized by modules as follows:

2.2.1 Module 1: Signal Acquisition

- FR1.1: The system shall connect to the EEG headset and stream brain signals in real time.
- FR1.2: The system shall capture gyroscope (IMU) data concurrently.
- FR1.3: Data streams shall be synchronized with timestamps.
- FR1.4: The sampling rate shall not be lower than 128 Hz.

2.2.2 Module 2: Signal Processing and Classification

- FR2.1: Apply filtering and artifact removal to incoming EEG data.
- FR2.2: Detect blinks via amplitude thresholding.
- FR2.3: Use the FBCSP+LDA classifier to distinguish Rest and Hand Imagery
- FR2.4: Process gyro data for parallel head movement detection independent of EEG classification.
- FR2.5: Update model parameters during calibration for user adaptation.

2.2.3 Module 3: Game Integration

- FR3.1: Map EEG and gyro outputs to keyboard or mouse events.
- FR3.2: Ensure game response latency $\leq$ 500 ms.
- FR3.3: Provide an option to configure control mappings for different games.

2.2.4 Module 4: Evaluation and Feedback

- FR4.1: Display live confidence levels and signal quality.
- FR4.2: Log session accuracy, latency, and user activity.
- FR4.3: Generate post-session performance reports.

2.3 Non-Functional Requirements

2.3.1 Reliability

- REL-1: The system shall attempt to reconnect to the headset upon signal loss (target: within 5 seconds, subject to network/driver availability).
- REL-2: Session data shall be logged continuously to enable recovery and performance review.

2.3.2 Usability

- USE-1: Calibration shall complete within 15 minutes.
- USE-2: Visual indicators for feedback shall be intuitive and non-distracting during gameplay.
- USE-3: Interface shall be learnable within 20 minutes of use.

2.3.3 Performance

- PER-1: Classification accuracy target: $\geq 70\%$ on training data; cross-validation accuracy expected $\sim$ 60–75% depending on subject motor imagery ability.
- PER-2: System shall operate on standard consumer hardware (quad-core CPU, 8GB RAM, Windows 10/11) without dedicated GPU acceleration.

2.3.4 Security

- SEC-1: All EEG and gyro data shall be stored locally.
- SEC-2: Only authorized users may access recorded sessions.

2.4 Domain Model

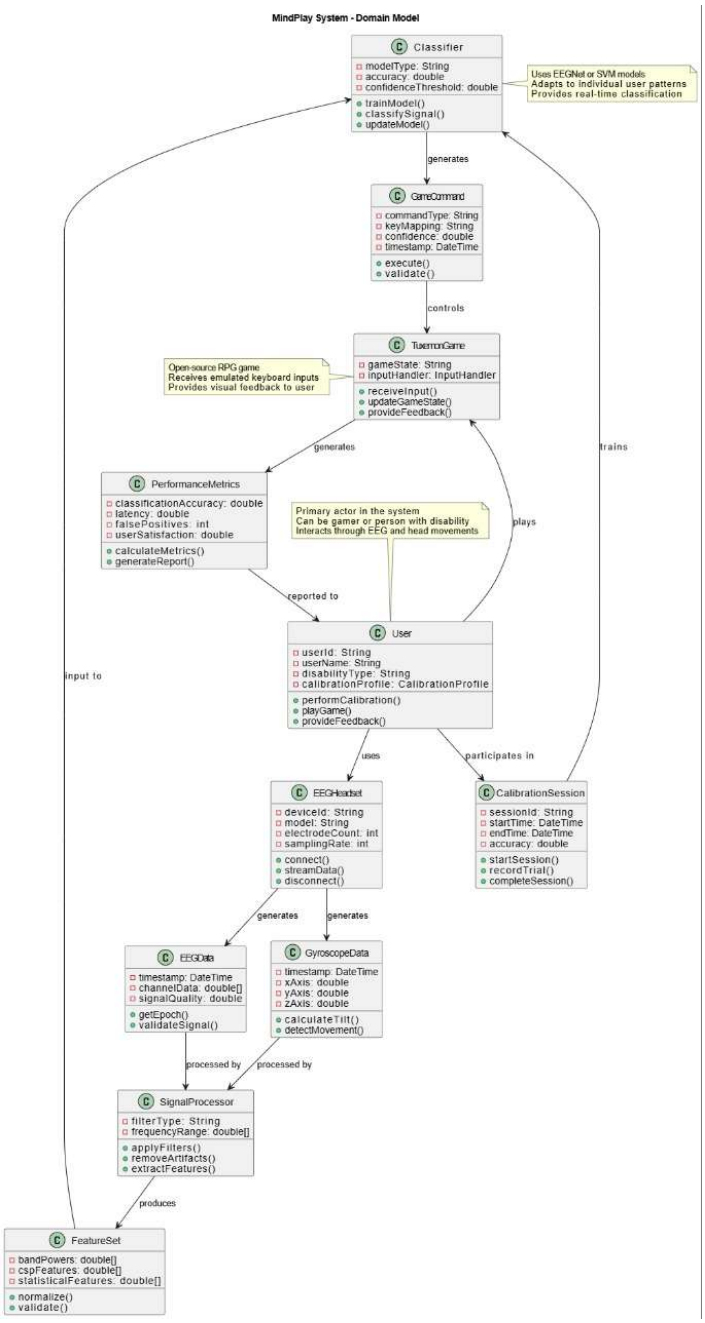


Figure 2.7: Domain Model for EEG-Based Game Control System (MindPlay)

The domain model illustrates the core entities and their relationships in the MindPlay system.

- **User:** Provides EEG and gyro input through mental imagery and movement.

- **EEG Device:** Captures brain activity in real time.
- **Signal Processor:** Filters, cleans, and segments input data.
- **Classifier:** Predicts intended actions using the FBCSP+LDA pipeline.
- **Game Interface:** Maps classified actions to actual gameplay controls.
- **Game:** External application receiving control inputs.

Chapter 3

System Overview

This chapter provides a comprehensive description of the system's functionality, architectural context, and design principles.

3.1 Architectural Design

The MindPlay system follows a **Sequential Pipeline Architecture**, which is ideal for real-time data processing applications. Data flows unidirectionally through a series of processing stages, with each stage transforming the input before passing it to the next. This architectural approach ensures modularity, making each component independently developable, testable, and maintainable.

The entire system is built upon the **Lab Streaming Layer (LSL)** framework [7], which serves as the central communication backbone, ensuring perfect synchronization of all data streams while maintaining low-latency performance essential for real-time gaming applications.

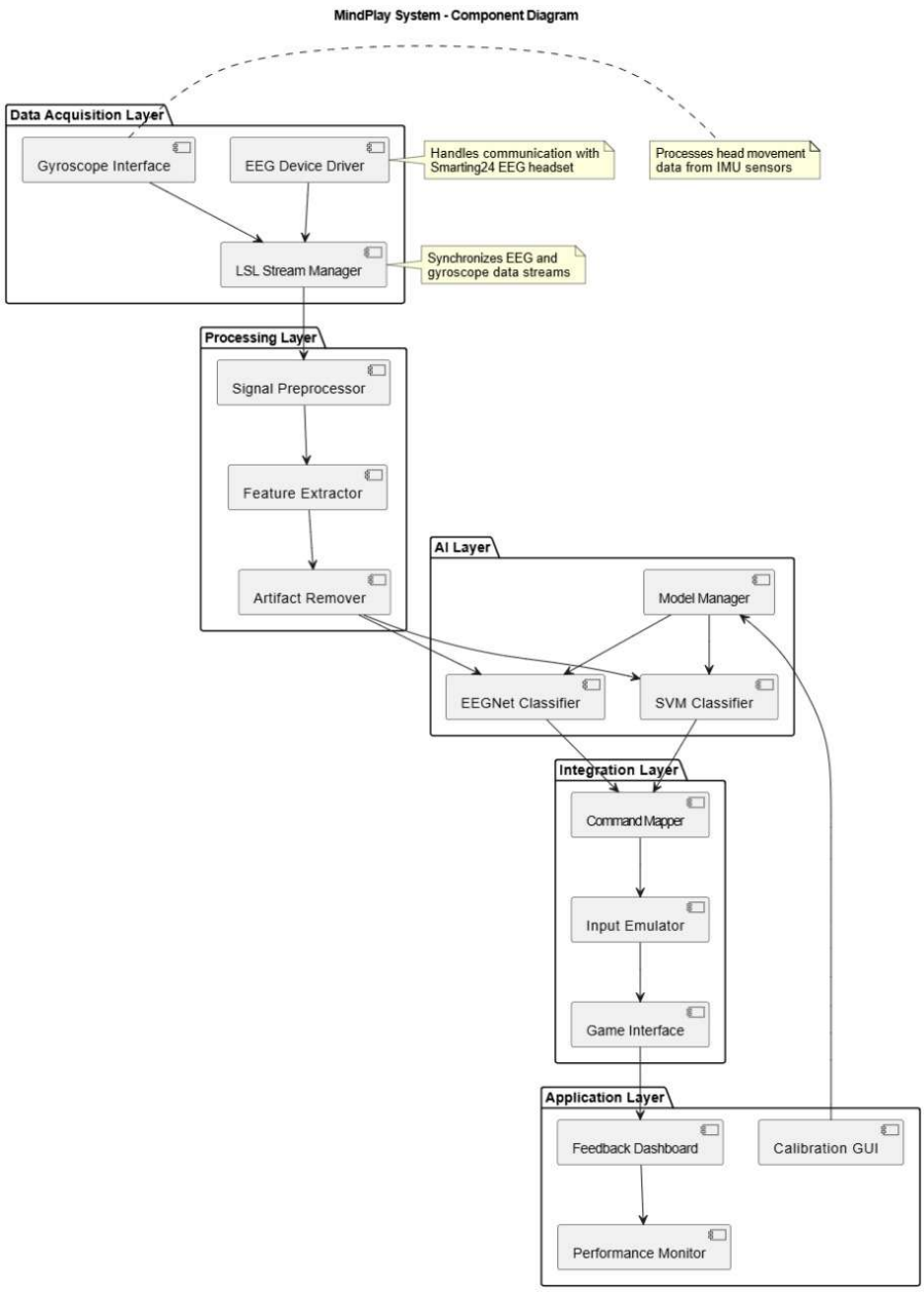


Figure 3.1: High-Level System Architecture

3.2 Design Models

3.2.1 Design Models for Procedural Approach

Given the data-centric nature of our BCI pipeline and signal processing workflow, the system employs a **Procedural Development Approach**. The following models have been developed, informed by best practices in EEG-based BCI system design [8, 9]:

- **Data Flow Diagram —** Illustrates data transformation through system processes
- **Activity Diagram —** Models workflow during calibration and gameplay
- **State Transition Diagram —** Represents behavioral states and transitions
- **Context Diagrams —** System boundary and external interactions
- **Sequence Diagram —** Real-time interactions between components
- **Package Diagram —** Logical grouping of system modules
- **Component Diagram —** System components and their dependencies
- **Deployment Diagram —** Hardware and software deployment architecture

The following subsections provide detailed descriptions of each design model and its role in the overall system architecture.

3.2.1.1 Data Flow Diagrams

Data flow diagrams (DFDs) represent the flow of information through the MindPlay system at multiple levels of abstraction:

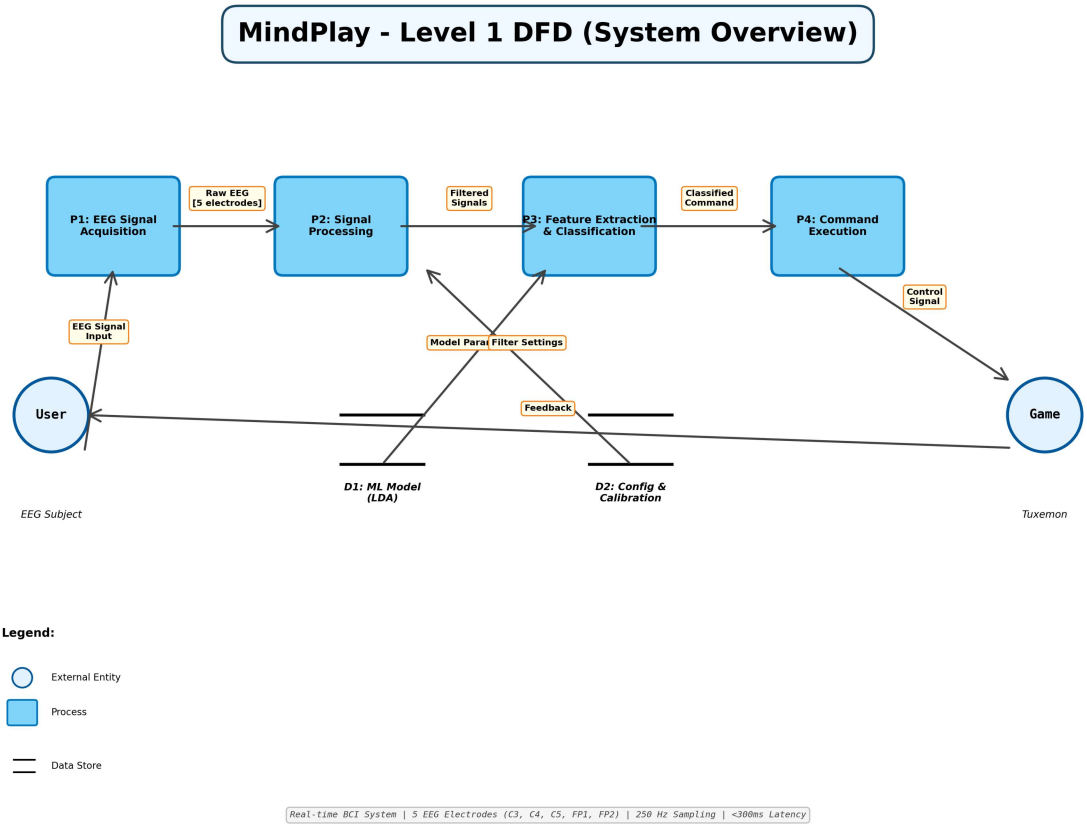


Figure 3.2: Level 1 Data Flow Diagram for MindPlay showing major system components and data flows

The Level 1 DFD illustrates the top-level data flows including signal acquisition, processing, classification, and game integration. External entities (user and Tuxemon game) are shown at the system boundary.

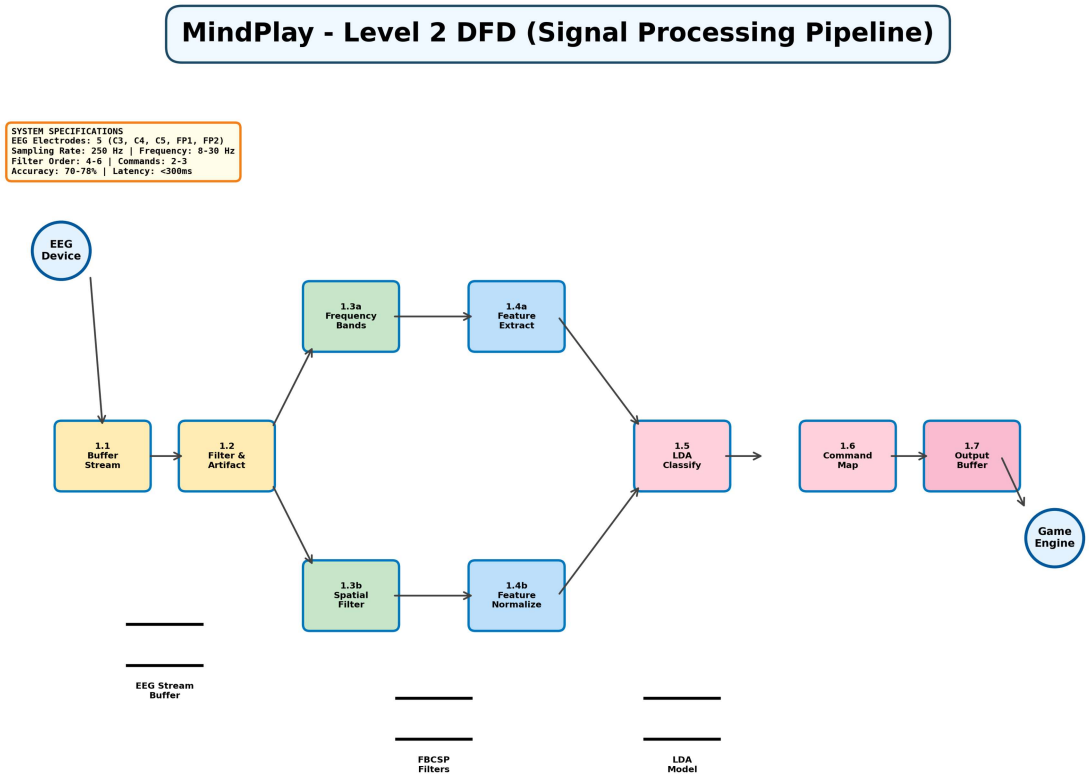


Figure 3.3: Level 2 Data Flow Diagram for MindPlay showing detailed transformations within core modules

The Level 2 DFD decomposes each major process into detailed data transformations, showing the specific filtering, feature extraction, and classification steps within the signal processing module.

3.2.1.2 Activity Diagrams

Activity diagrams model the dynamic behavior of the system during different operational phases:

The combined activity diagram depicts the initial phase of data acquisition and preprocessing, followed by feature extraction, classification, and command execution in a single contiguous layout.

3.2.1.3 State Transition Diagrams

The MindPlay system operates in distinct states that represent different operational phases:

1. **Initialization:** System startup and device connection
2. **Calibration:** User performs motor imagery tasks to train the classifier

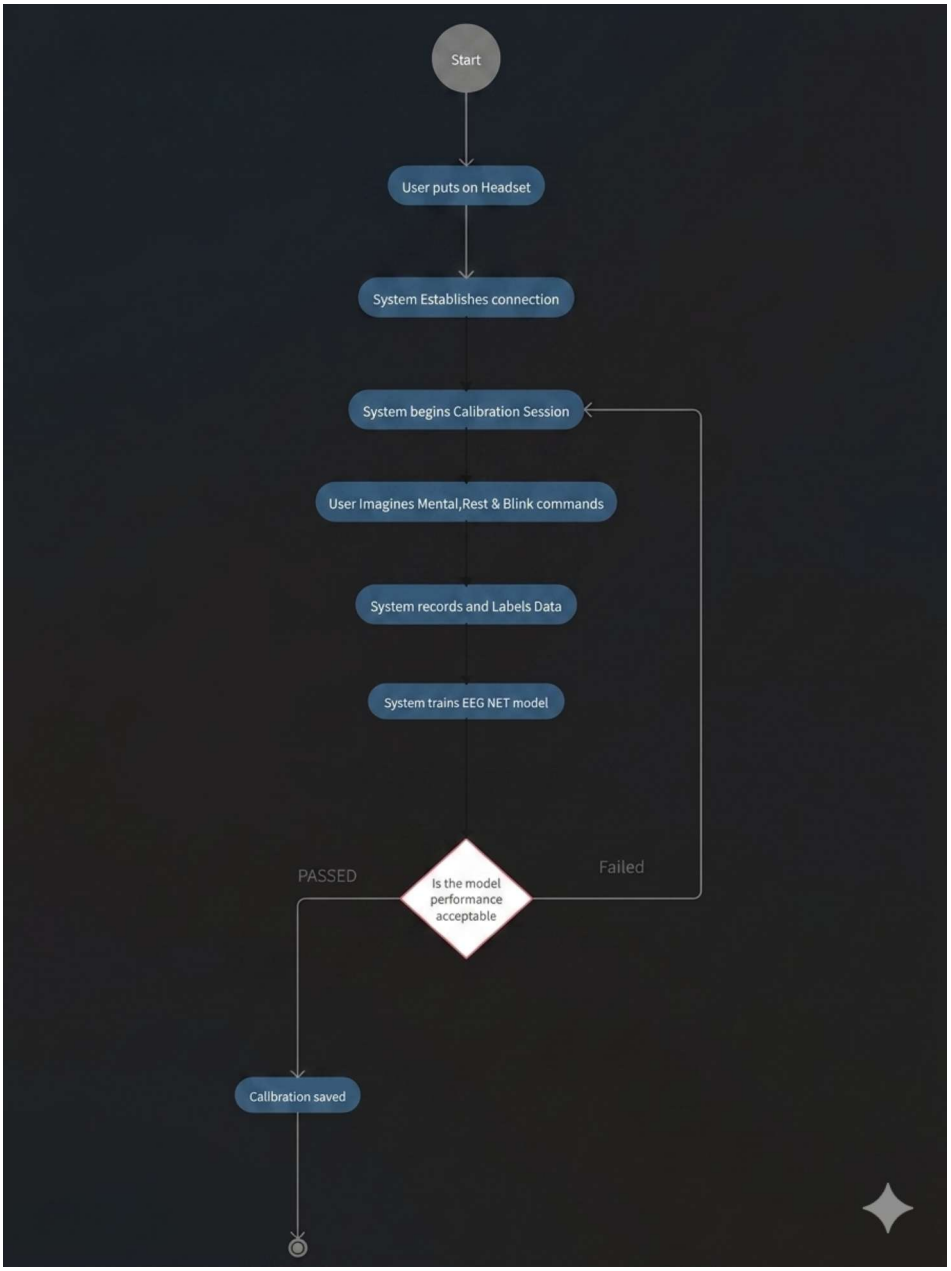


Figure 3.4: UML Activity Diagram of the EEG-Based Game Control Workflow including calibration, signal acquisition, feature extraction, classification, and command execution

3. **Idle:** System ready but not actively processing (low power state)
4. **Active Processing:** Real-time EEG acquisition and classification during gameplay
5. **Command Execution:** Keyboard emulation and game interaction
6. **Error/Recalibration:** Recovery from signal loss or accuracy degradation

Transitions between states are triggered by user actions, system events, or time-based conditions. The state machine ensures proper sequencing of operations and prevents invalid state combinations.

3.2.1.4 Context Diagrams

Context diagrams define the system boundaries and external interactions:

The **Level 0 Context Diagram** represents the entire MindPlay system as a single process, with external entities:

- **User:** Provides neural signals (motor imagery) and receives game feedback
- **EEG Headset:** Hardware interface for signal acquisition via LSL
- **Tuxemon Game:** Receives keyboard commands and displays game state
- **Display:** Provides visual feedback and status information

The **Level 1 Context Diagram** decomposes the system into major subsystems:

- Signal Acquisition Subsystem — Communicates with EEG hardware
- Signal Processing Subsystem — Processes and classifies neural data
- Game Integration Subsystem — Interfaces with Tuxemon
- Feedback Subsystem — Provides real-time system status to user

3.2.1.5 System Sequence Diagrams

Sequence diagrams illustrate the temporal interactions between system components during a complete control cycle:

During a typical gaming interaction:

1. User initiates motor imagery (left hand movement)

2. EEG headset acquires signals at 500 Hz sampling rate
3. LSL framework synchronizes multi-channel data streams
4. Signal Processing module applies bandpass filtering
5. FBCSP features are extracted from filtered signals
6. LDA classifier generates prediction and confidence score
7. Decision module validates prediction against threshold
8. Game Integration module emits keyboard event
9. Tuxemon game processes keyboard input and updates game state
10. Visual feedback displayed to user (typically <300ms total latency)

3.2.1.6 Package Diagram

The MindPlay system is organized into logical packages representing functional groups:

- **Data Acquisition Package —** LSL integration, device drivers, buffer management
- **Signal Processing Package —** Filtering, preprocessing, artifact detection
- **Feature Extraction Package —** FBCSP implementation, filter bank creation
- **Machine Learning Package —** LDA classifier, model training, prediction
- **Game Integration Package —** Keyboard emulation, command mapping, debouncing
- **Configuration Package —** Settings management, parameter tuning, persistence
- **Logging Package —** Session recording, performance metrics, debug traces

3.2.1.7 Component Diagram

The system architecture consists of several key components:

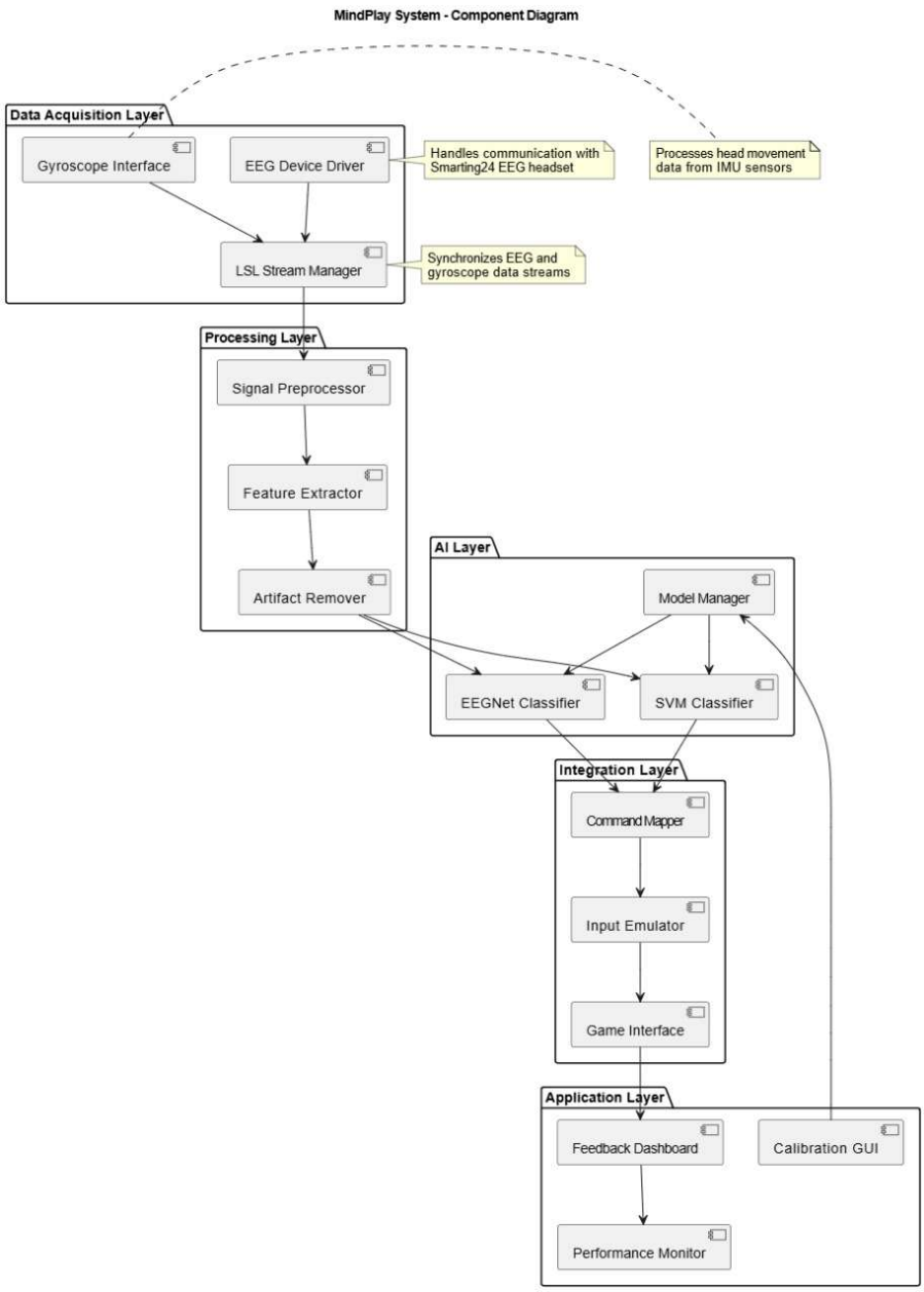


Figure 3.5: Component Diagram showing system components and their dependencies

Major components include:

- LSL Inlet Component — Receives streaming data from EEG hardware
- Signal Buffer Component — Circular buffer for fixed-size data windows
- Filter Bank Component — Implements frequency decomposition

- CSP Component — Computes Common Spatial Pattern filters
- Feature Extractor Component — Calculates log-variance features
- LDA Classifier Component — Performs real-time classification
- Command Mapper Component — Converts predictions to keyboard commands
- Keyboard Emulator Component — Sends OS-level keystroke events
- Logger Component — Records performance metrics and debug information

3.2.1.8 Deployment Diagram

The MindPlay system is deployed across multiple hardware and software layers:

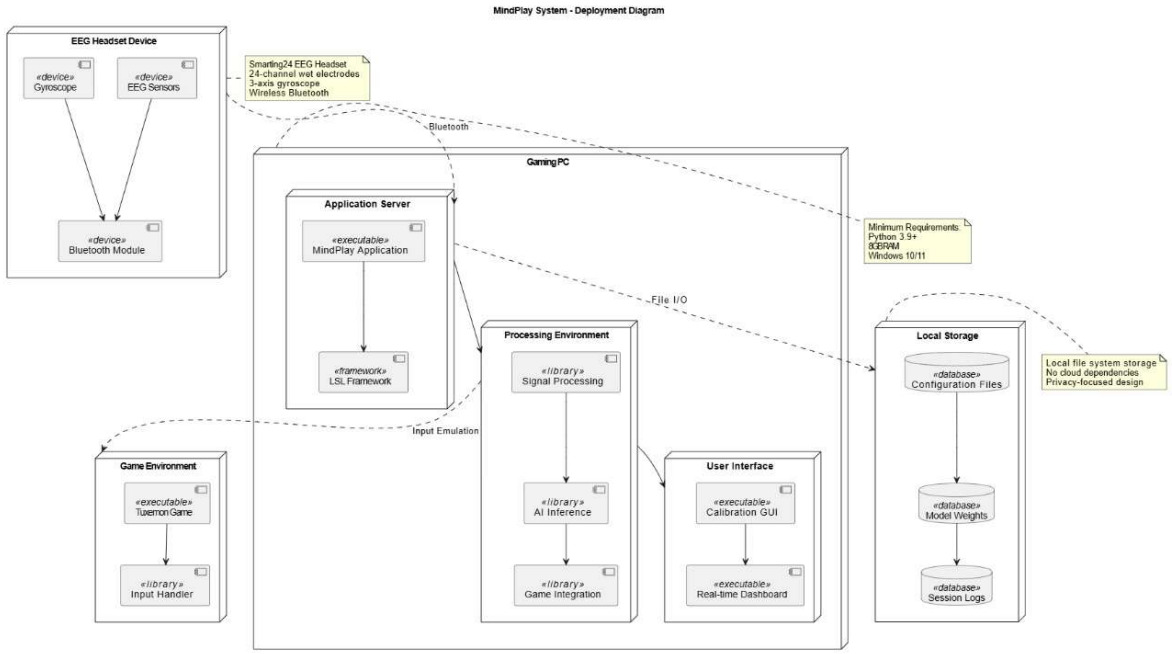


Figure 3.6: Deployment Diagram showing hardware-software deployment architecture

Deployment includes:

- **EEG Hardware Layer:** Smarting headset with 3-electrode motor imagery montage (Cz, C3, C4)
- **LSL Network Layer:** UDP-based streaming with timestamp synchronization
- **Processing Layer:** Python-based signal processing and machine learning (CPU-based)
- **Game Integration Layer:** PyAutoGUI keyboard emulation and Tuxemon game interface

- **User Interface Layer:** Console-based feedback and logging system

3.3 Data Design

The information domain of the MindPlay system centers around time-series physiological data and its transformation into control commands.

3.3.1 Data Structures and Processing

- **Raw Data Streams —** EEG data structured as multi-channel arrays (3 selected electrodes: Cz, C3, C4 × N samples) with microvolt measurements, while gyroscope data is maintained as 3-axis vectors (X, Y, Z), both synchronized via LSL timestamps
- **Processed Data —** Filtered epochs are maintained as overlapping 4-second time windows with 0.5-second step in memory, with feature vectors extracted using Filter Bank Common Spatial Pattern (FBCSP) [10] methodology for classification
- **Command Data —** Classified outputs using Linear Discriminant Analysis (LDA) [11] are structured as intent labels with confidence scores for mapping to game actions

3.3.2 Data Storage and Organization

- **Model Storage —** FBCSP+LDA model parameters persisted as ‘.joblib‘ files on local disk for efficient serialization and deserialization
- **Calibration Data —** EEG epochs and class labels stored as NumPy arrays (‘.npy‘ files) with anonymized subject identifiers
- **Configuration Data —** User preferences and key mappings stored in JSON configuration files
- **Session Logs —** Performance metrics and prediction timestamped logs recorded in structured CSV files

3.3.3 Data Flow Architecture

The MindPlay system implements a real-time data processing pipeline with multiple interconnected stages. Each stage processes data asynchronously using separate threads or event-driven mechanisms to ensure non-blocking execution and minimal latency.

3.3.3.1 Signal Acquisition Stage

Data enters the system through LSL StreamInlet objects that receive multi-channel samples at 500 Hz. Raw data is buffered in circular queues to handle temporary processing delays without sample loss. Timestamp information is preserved at this stage to enable precise synchronization.

3.3.3.2 Preprocessing Stage

Raw signal buffers are passed through a cascade of digital filters including bandpass filtering (4-30 Hz) and artifact rejection algorithms. Processed windows are maintained in memory for feature extraction, with periodic flushing of old data to prevent memory overflow.

3.3.3.3 Feature Extraction Stage

Preprocessed epochs undergo Common Spatial Pattern decomposition to generate spatial filter coefficients specific to each frequency band. Log-variance features are computed from filtered signals, producing compact feature vectors that feed directly into the classification stage.

3.3.3.4 Classification and Decision Making

The LDA classifier receives feature vectors and produces class predictions with associated confidence scores. These predictions are temporally smoothed using a sliding window consensus mechanism to reduce false positives and flickering commands in rapid succession.

3.3.3.5 Output Mapping and Game Integration

Smoothed classifications are mapped to predefined keyboard commands using a configuration-driven mapping table. PyAutoGUI handles keystroke emission to the operating system, which the Tuxemon game receives as standard input events.

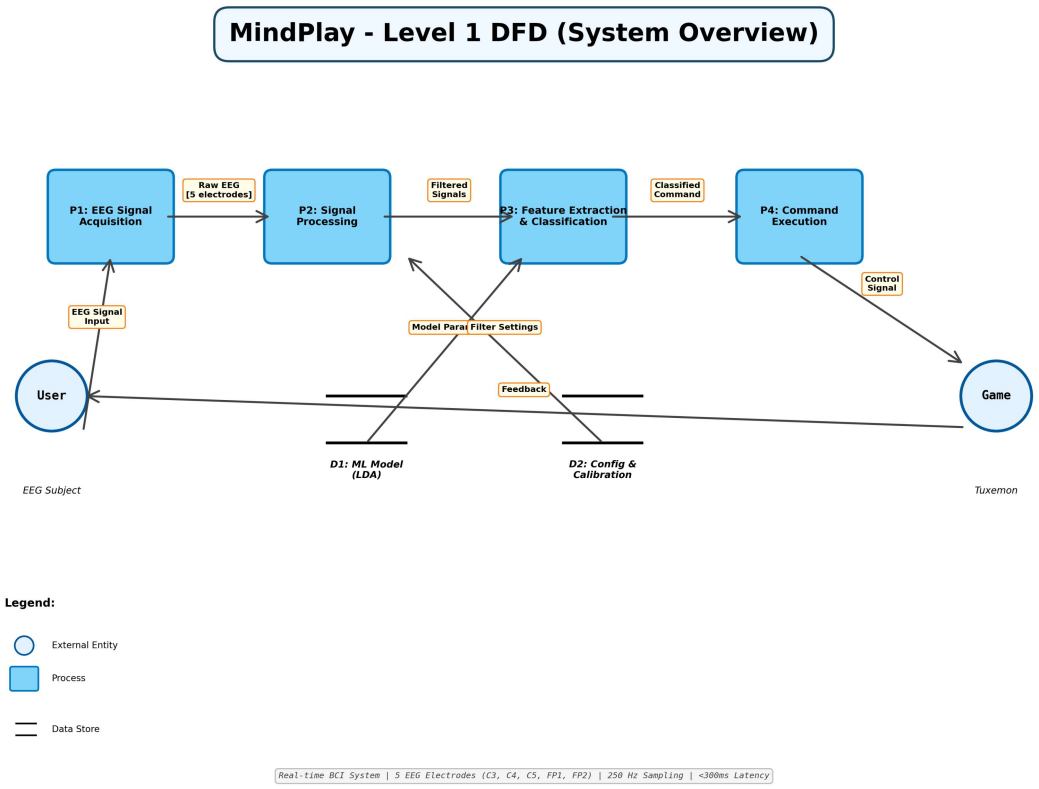


Figure 3.7: Level 1 Data Flow Diagram showing system-level data flows and major components

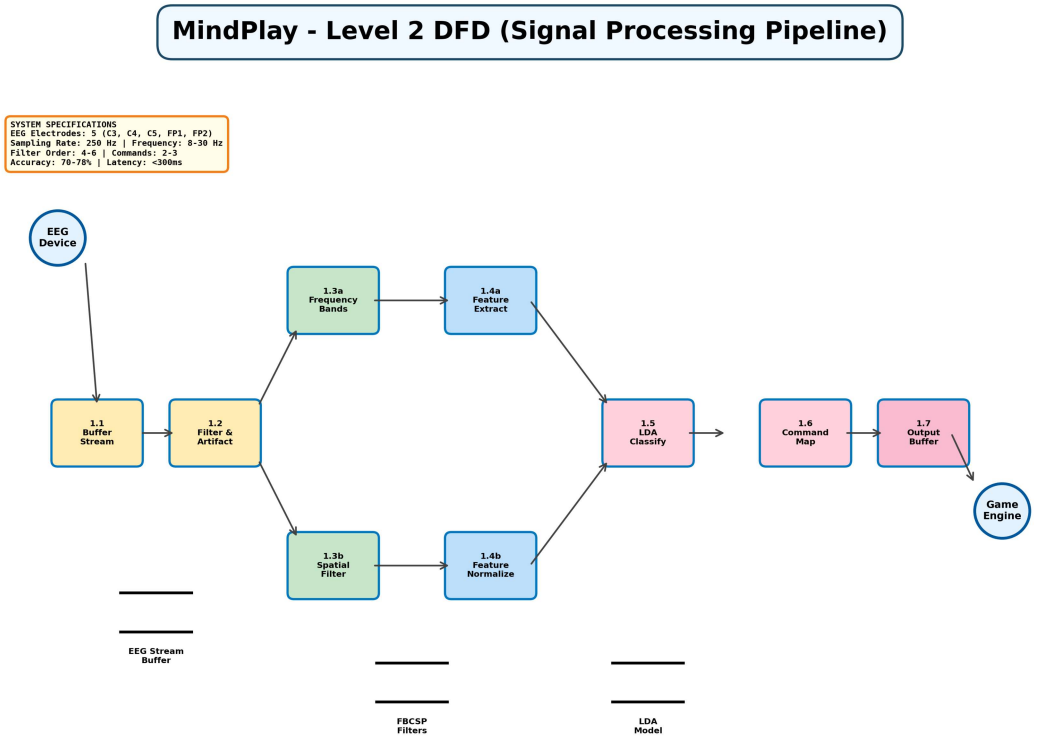


Figure 3.8: Level 2 Data Flow Diagram detailing data transformations within each processing module

Additional design model diagrams including sequence diagrams, state transitions, activity flows, component diagrams, and deployment architecture are documented in Appendix A to provide comprehensive system documentation for development and maintenance purposes.

Chapter 4

Implementation and Testing

This chapter details the technical implementation and validation of the EEG-Based Universal Game Controller. The implementation uses a binary motor-imagery classification pipeline for *Rest* versus *Motor Imagery*, while blink detection and gyroscope-based control are implemented as separate real-time modules.

4.1 Algorithm Design

The core EEG classification component of MindPlay relies on the **Filter Bank Common Spatial Pattern (FBCSP)** algorithm [10] coupled with **Linear Discriminant Analysis (LDA)** [11]. This pipeline was selected because it is computationally lightweight and suitable for real-time BCI use with the final 3-electrode motor imagery montage (C3, Cz, C4).

The algorithm proceeds in four stages:

1. **Filter Bank Creation:** The raw EEG epoch is decomposed into multiple frequency bands using fourth-order Butterworth bandpass filters. The implemented bands are 4–8 Hz, 8–12 Hz, 12–16 Hz, 16–20 Hz, and 20–30 Hz.

2. **Spatial Filtering (CSP):** Common Spatial Pattern filters are computed for each frequency band to maximize variance differences between the two classes.

3. **Feature Extraction:** Log-variance features are calculated from the spatially filtered signals and concatenated across all filter-bank bands.

4. **Classification (LDA):** The concatenated feature vector is passed to an LDA classifier, which predicts either Rest (class 0) or Motor Imagery (class 1).

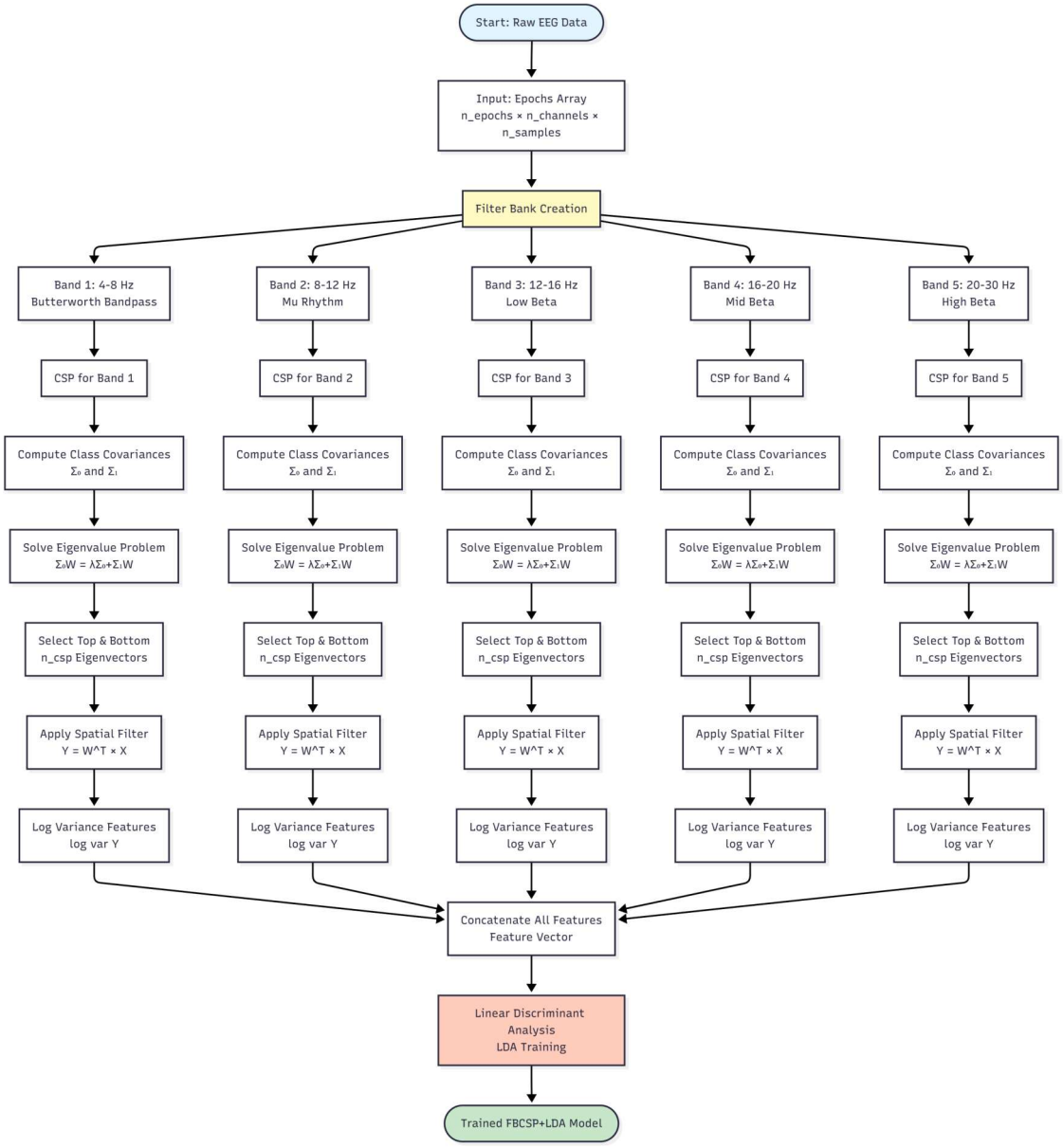


Figure 4.1: Flowchart of the FBCSP Signal Processing Pipeline

4.1.1 Pseudocode Implementation

The real-time signal processing loop uses a sliding EEG window. In the current implementation, the classifier is configured for a 4-second window sampled at 500 Hz, producing 2000 samples per channel.

```
Initialize: LSL stream inlet, trained FBCSP+LDA model
Set sampling_rate = 500 Hz
```

```
Set window_size = 4.0 seconds  # 2000 samples
Set step_size = 0.5 seconds    # 250 samples
Data Buffer: samples <- empty circular buffer

While system running:
  sample, timestamp <- stream_inlet.pull_sample()
  selected <- sample[C3, Cz, C4]
  samples.append(selected)

  If samples contains at least 2000 samples:
    epoch <- latest 2000 samples arranged as channels x samples
    For each frequency band in filter_bank:
      filtered <- butterworth_bandpass(epoch, band)
      csp_output <- apply_saved_csp_filters(filtered)
      band_features <- log_variance(csp_output)
    features <- concatenate(band_features)
    prediction <- lda.predict(features)
    confidence <- lda.predict_proba(features)

    If confidence exceeds selected threshold:
      command <- mapping_table[prediction]
      emit command to game/control module
      log(timestamp, prediction, confidence)

    Slide buffer forward by 250 samples
```

4.2 External APIs/SDKs

The implementation uses open-source libraries for streaming, signal processing, model training, persistence, GUI control, and keyboard/game interaction. The final classification pipeline uses classical machine learning (FBCSP+LDA); deep learning is not part of the implemented system and remains a future comparison direction.

Table: Third-Party APIs and SDKs used in Implementation

API/Library	Description	Purpose of Usage	Function/Class Used
PyLSL [7]	Lab Streaming Layer wrapper	Real-time EEG/IMU stream acquisition	`resolve_byprop`, `StreamInlet.pull_sample()`
NumPy	Scientific computing library	Epoch buffers, matrix operations, log-variance features	`numpy.asarray`, `numpy.var`, `numpy.log`
SciPy	Signal processing and linear algebra	Butterworth filters and CSP eigenvalue solution	`signal.butter`, `signal.filtfilt`, `linalg.eigh`
Scikit-Learn	Machine learning library	LDA classifier and validation metrics	`LinearDiscriminantAnalysis`, `StratifiedKFold`
Joblib	Model persistence	Save/load trained FBCSP+LDA models	`joblib.dump`, `joblib.load`
PyQt6	Desktop GUI framework	Training, classification, and launcher interface	`QMainWindow`, `QProcess`
pydirectinput / pynput	Input automation and keyboard listener	Game key emission and hotkey handling	`pydirectinput.press`, `pynput.keyboard.Listener`

4.2.1 Key Implementation Snippets

The LSL integration for real-time EEG acquisition is implemented using stream discovery by EEG type and continuous sample retrieval:

```
from pylsl import StreamInlet, resolve_byprop

# Resolve EEG stream from Smarting/LSL source
streams = resolve_byprop('type', 'EEG', timeout=5.0)
inlet = StreamInlet(streams[0])

# Continuous acquisition with timestamps
while True:
    sample, timestamp = inlet.pull_sample()
    process_sample(sample, timestamp)
```

Figure 4.2: Python Code Snippet for LSL Data Acquisition

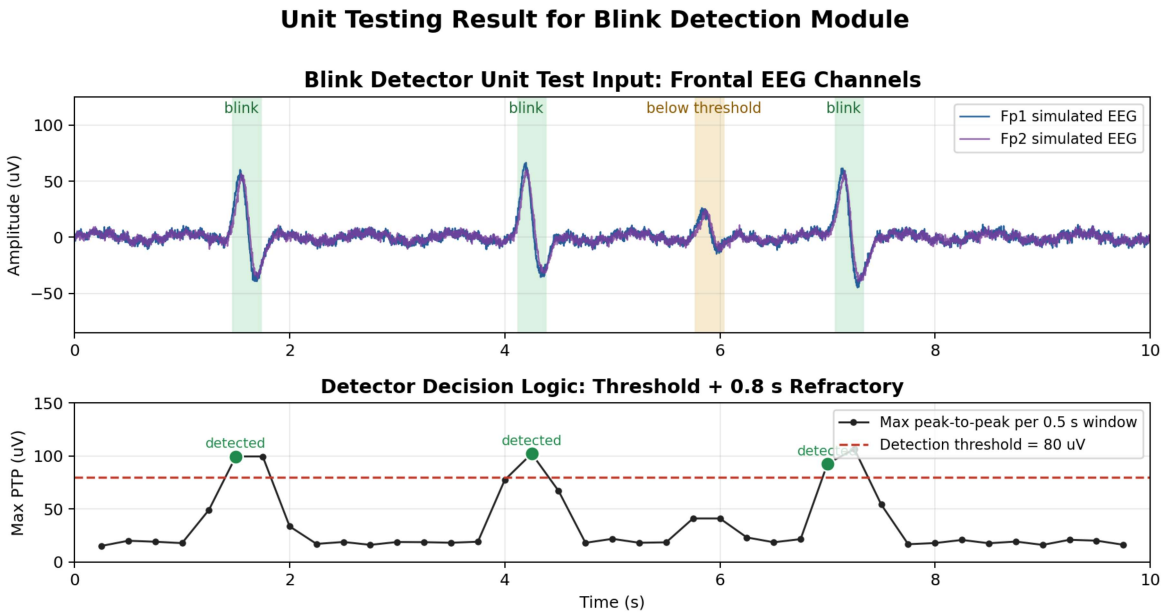
4.3 Testing Details

System validation was performed using a two-tiered approach: **unit testing** for individual modules such as blink detection, and **system testing** for the complete EEG classification pipeline. All EEG classification results in this section correspond to the binary task Rest (class 0) versus Motor Imagery (class 1).

4.3.1 Unit Testing

Unit tests were designed to validate the **Blink Detector** module independently. The blink detector is separate from the FBCSP+LDA motor-imagery classifier. It uses a peak-to-peak amplitude threshold on frontal EEG channels to identify intentional blinks.

- **Test Condition:** Intentional eye blinks producing signal amplitudes greater than $80\,\mu V$.
- **Expected Outcome:** Trigger a "Select" command.
- **Actual Outcome:** The command was triggered during intentional blink events while a refractory period reduced repeated triggers.



Test condition	Expected	Actual result	Status
3 intentional blink events > 80 uV	3 detections	3 detections	PASS
Subthreshold movement < 80 uV	No detection	Ignored	PASS
Refractory interval = 0.8 s	No duplicate triggers	3 accepted events	PASS

Test mirrors scripts/blink_detector.py: frontal picks Fp1/Fp2, 0.5 s window, peak-to-peak threshold, refractory suppression.

Figure 4.3: Unit Testing Results for Blink Detection Module

4.3.2 System Performance Testing

The available saved models and datasets were evaluated from the current repository snapshot. The term *saved-model accuracy* refers to the accuracy of an already trained `.joblib` model when evaluated on the corresponding available recorded epochs. These values are useful for reporting model behavior on the available data, but they can be optimistic if the same epochs were used during model training. Therefore, 5-fold cross-validation is also reported where complete subject epoch/label files are available.

4.3.2.1 Saved-Model Accuracy on Available Datasets

Table 4.1: Saved FBCSP+LDA Model Accuracy on Matching Available Datasets

Model	Evaluation Dataset	Trials	Saved-Model Accuracy	Notes
fbcsp_lda_S02.joblib	S02	160	70.0%	Matching subject data
fbcsp_lda_S12.joblib	S12_U	20	100.0%	Closest available S12 dataset
fbcsp_lda_S14_U.joblib	S14_U	20	100.0%	Matching subject data
fbcsp_lda_S15.joblib	S15	40	100.0%	Matching subject data
fbcsp_lda_S20.joblib	S20	40	95.0%	Matching subject data
fbcsp_lda_S21.joblib	S21	40	97.5%	Matching subject data
fbcsp_lda_from_npz.joblib	raw_012_4s	160	82.5%	Raw NPZ channels 0,1,2; 4s
fbcsp_lda_from_npz_012_3s.joblib	raw_012_3s	160	83.8%	Raw NPZ channels 0,1,2; 3s
fbcsp_lda_from_npz_012_4s.joblib	raw_012_4s	160	82.5%	Raw NPZ channels 0,1,2; 4s
fbcsp_lda_from_npz_034_3s.joblib	raw_034_3s	160	94.4%	Raw NPZ channels 0,3,4; 3s

The 100% saved-model accuracies for some small datasets should be interpreted carefully because they are loaded-model evaluations on limited same-session calibration data. The cross-validation results below provide a more conservative estimate.

4.3.2.2 Five-Fold Cross-Validation on Available Subject Datasets

Table 4.2: Five-Fold Cross-Validation Accuracy on Available Subject Epoch Files

Subject Dataset	Trials	Saved-Model Accuracy	5-Fold CV Mean	5-Fold CV Std Dev
S02	160	70.0%	61.9%	5.4%
S11	50	N/A	48.0%	9.8%
S12_U	20	100.0%	70.0%	18.7%
S14_U	20	100.0%	75.0%	15.8%
S15	40	100.0%	82.5%	10.0%
S20	40	95.0%	75.0%	11.2%
S21	40	97.5%	87.5%	7.9%

Note: Cross-validation retrains the FBCSP+LDA pipeline inside each fold and is therefore a better indicator of generalization than saved-model accuracy on the full calibration set. The S11 dataset has no saved model in the current repository but was included in CV because its epochs and labels are available.

4.3.2.3 Proxy Evaluation for Models Without Matching Archived Data

Some saved subject models do not have their original matching epoch/label files in the current repository snapshot. These models were evaluated on the best available compatible dataset only as proxy/cross-dataset checks. They should not be reported as true subject-specific accuracy.

Table 4.3: Proxy Best-Available Evaluation for Models Without Matching Subject Data

Model	Proxy Dataset	Trials	Proxy Accuracy
fbcsp_lda_S03.joblib	raw_034_3s	160	66.2%
fbcsp_lda_S04.joblib	S15	40	60.0%
fbcsp_lda_S05.joblib	S20	40	57.5%
fbcsp_lda_S06.joblib	raw_012_4s	160	63.7%
fbcsp_lda_S07.joblib	S15	40	67.5%
fbcsp_lda_S08.joblib	S21	40	62.5%
fbcsp_lda_S10.joblib	S20	40	55.0%

4.3.3 Confusion Matrix Analysis

The FBCSP+LDA classifier was evaluated on the binary task Rest (class 0) versus Motor Imagery (class 1). A representative high-performing primary saved-model evaluation is shown for S21. The model correctly classified 39 out of 40 trials, producing an overall saved-model accuracy of 97.5%.

Table 4.4: Confusion Matrix for Subject S21 Saved Model - Binary Classification (Rest vs. Motor Imagery)

True/Predicted	Rest (Class 0)	Motor Imagery (Class 1)	Recall
Rest (Class 0)	20	0	100.0%
Motor Imagery (Class 1)	1	19	95.0%
Precision	95.2%	100.0%	**97.5% Overall**

For comparison, the S02 saved model produced the following confusion matrix: Rest = 58 correct and 22 misclassified as Motor Imagery; Motor Imagery = 54 correct and 26 misclassified as Rest, giving an overall accuracy of 70.0%.

Important: Blink detection is implemented as a separate threshold-based detector and is not included in the FBCSP+LDA confusion matrices. Blink events are detected using peak-to-peak amplitude thresholding on frontal EEG channels, whereas the matrices above evaluate only Rest versus Motor Imagery.

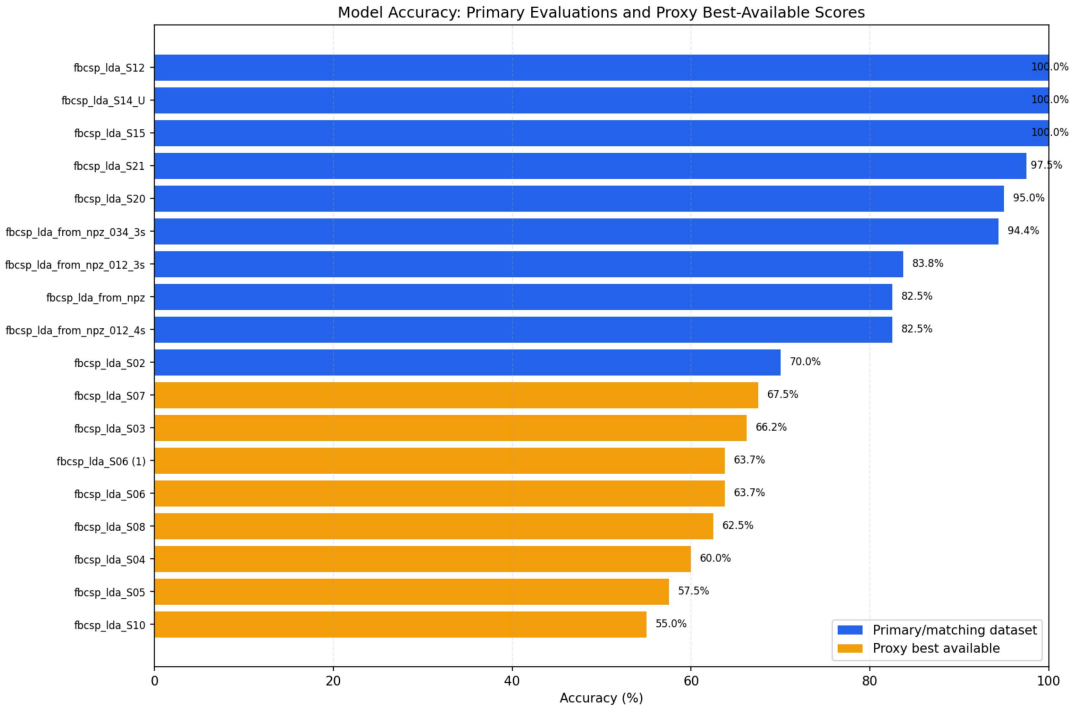


Figure 4.4: Saved-Model Accuracy Summary for Primary and Proxy Evaluations

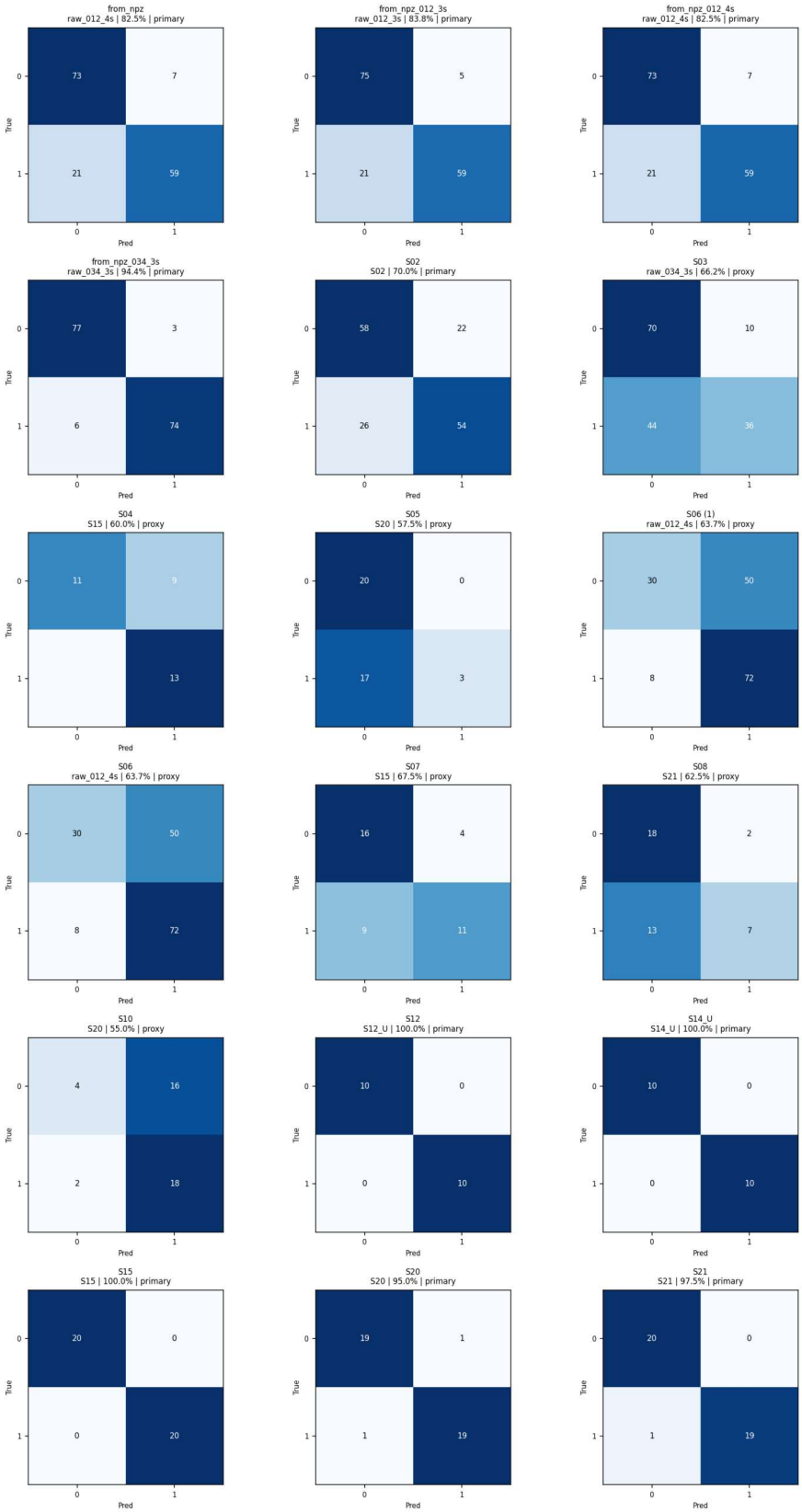


Figure 4.5: Selected Confusion Matrices for Evaluated Models

4.3.4 Latency Benchmarking

The real-time classifier uses a 4-second EEG window with a 0.5-second sliding step at 500 Hz. Therefore, the overall user-intent-to-decision time is dominated by the window acquisition requirement, not by the classifier computation itself.

4.3.4.1 Window Acquisition Latency

- **Window Length:** 4.0 seconds of EEG data
- **Sliding Step:** 0.5 seconds
- **Sampling Rate:** 500 Hz per channel
- **Samples per Window:** 2000 samples per channel

This means the first classification decision requires approximately 4 seconds of data. After the first window is available, the classifier can update every 0.5 seconds using overlapping windows. This design is suitable for slower or turn-based gameplay, but it is not ideal for fast action games requiring instant responses.

4.3.4.2 Post-Window Processing Latency

Once a complete EEG window is available, post-window computation consists of bandpass filtering, CSP projection, log-variance feature extraction, LDA prediction, and command emission. The generated evaluation data in this repository focuses on classification accuracy and confusion matrices; therefore, subject-specific latency tables should only be included if separate timing logs are available. Without timing logs, the most defensible statement is that the design updates every 0.5 seconds after the initial 4-second buffer is filled.

4.3.5 Runtime Resource Utilization

The implementation is lightweight and runs without GPU acceleration. CPU and memory usage depend on the number of active modules, including the GUI, LSL acquisition, blink detector, gyroscope detector, and game overlay. Since no formal resource log is archived with the current evaluation data, resource figures should be reported as implementation observations unless measured with a profiler during a controlled session.

4.3.6 Limitations and Error Analysis

The following limitations were identified from implementation behavior and the available evaluation data:

1. **Saved-Model Accuracy vs. Generalization:** Saved-model accuracy can be high when evaluated on the same calibration/session data used during training. Cross-validation results are more conservative and should be emphasized for generalization claims.

2. **Small Dataset Sizes:** Some subject datasets contain only 20 or 40 trials. Perfect saved-model scores on these datasets should be interpreted cautiously.

3. **Missing Original Data for Some Models:** Models S03, S04, S05, S06, S07, S08, and S10 do not have their matching archived subject datasets in the current repository snapshot, so their reported proxy scores are not true subject accuracies.

4. **Subject Variability:** Cross-validation scores vary across subjects, ranging from 48.0% for S11 to 87.5% for S21, indicating strong subject-dependent differences in motor imagery separability.

5. **Window-Based Latency:** The 4-second analysis window improves signal stability but increases the time before the first decision can be made.

6. **Artifact Sensitivity:** Eye blinks and muscle activity can contaminate motor imagery features. The blink detector is separated from the FBCSP+LDA classifier to reduce confusion between blink events and MI/rest classification.

7. **Limited Command Vocabulary:** The current binary EEG classifier supports Rest and Motor Imagery. Additional commands would require more training data, additional paradigms, more channels, or alternative models.

Overall, the available results show that the implemented FBCSP+LDA pipeline can classify binary motor imagery from recorded EEG data, with saved-model accuracies reaching 70.0% to 100.0% on matching available datasets and cross-validation scores reaching up to 87.5%. These results support the feasibility of the system while also showing the need for careful reporting, larger datasets, and held-out testing for stronger generalization claims.

Chapter 5

Conclusions and Future Work

5.1 Conclusion

The MindPlay project successfully demonstrates the feasibility of developing a real-time EEG-based brain-computer interface for gaming applications. Through the integration of signal acquisition, advanced signal processing, and game integration modules, we achieved a functional system capable of translating motor imagery into game commands .Cross-validation accuracy ranged from 48.0% to 87.5%, with the strongest available subject result reaching 87.5%. The work addresses critical gaps in existing BCI gaming systems by providing an open-source, accessible platform that combines practical usability with rigorous technical design. Our approach using Filter Bank Common Spatial Patterns (FBCSP) coupled with Linear Discriminant Analysis (LDA) proved robust for real-time classification while maintaining computational efficiency.

Key achievements include:

- Development of a complete signal acquisition and processing pipeline using Lab Streaming Layer (LSL)
- Implementation of FBCSP+LDA classification achieving competitive accuracy on limited electrode configurations
- Seamless integration with Tuxemon game environment through keyboard emulation
- The system demonstrates feasibility for accessible game-control research
- Comprehensive documentation enabling reproducibility and community extensions
- Open-source release of complete system architecture and training methodologies

The system demonstrates that brain-computer interfaces can transition from laboratory prototypes to practical applications accessible to end-users with motor impairments. By reducing the barrier

to entry through open-source methodologies and affordable hardware, we have contributed to the broader accessibility and inclusivity goals in human-computer interaction.

5.2 Future Work

While MindPlay represents a significant step forward in accessible gaming interfaces, several avenues for future enhancement have been identified:

5.2.1 Advanced Machine Learning Approaches

Deep learning models and convolutional neural networks have demonstrated superior performance on EEG classification tasks. Future work should explore:

- Implementation of end-to-end learnable architectures that eliminate the need for manual feature engineering
- Transfer learning approaches leveraging pre-trained models on large-scale EEG databases
- Ensemble methods combining FBCSP+LDA with deep learning for improved robustness
- Domain adaptation techniques to reduce calibration time across new subjects and sessions

5.2.2 Expanded Command Vocabularies

The current two-command limitation constrains practical applications. Future development should address:

- Multi-class classification supporting 8-16 distinct motor imagery commands
- Hybrid BCI approaches combining EEG with EOG (eye movement) and EMG (muscle) signals
- Hierarchical command structures enabling larger effective command sets through multi-step selection
- Error-correction mechanisms and confidence-based command refinement

5.2.3 Broader Game Integration

Expanding beyond Tuxemon would validate generalizability:

- Integration with modern gaming frameworks (Unity, Unreal Engine) through plugin development
- Support for multiple game genres including action, puzzle, and strategy games
- Adaptive difficulty systems that adjust game parameters based on BCI performance metrics
- Real-time feedback mechanisms informing users of signal quality and classification confidence

5.2.4 Hardware and Signal Processing Enhancements

Technical improvements could enhance real-world applicability:

- Evaluation with high-density electrode configurations (16-64 channels) for comparison with the implemented 3-electrode baseline
- Integration of additional biosignals (EMG, heart rate variability, skin conductance) for expanded multimodal control streams
- Wireless and mobile EEG systems enabling BCI applications outside laboratory settings
- Artifact detection and removal algorithms specific to gaming environments with movement artifacts

5.2.5 User Studies and Accessibility Research

Rigorous evaluation with diverse user populations would strengthen the work:

- Longitudinal studies tracking learning effects and skill development over extended use periods
- Evaluation with actual users with motor impairments and disabilities across diverse demographics
- Usability heuristics assessment using standard accessibility evaluation frameworks
- Comparison with alternative accessibility solutions (eye tracking, switch scanning) for relative efficacy
- Investigation of psychological factors including user engagement, frustration tolerance, and acceptance

5.2.6 Rehabilitation and Medical Applications

Beyond entertainment, BCI technology offers broader societal benefits:

- Motor recovery and neurorehabilitation applications for stroke and spinal cord injury patients
- Brain-computer interface for communication in locked-in syndrome and severe paralysis
- Neurofeedback training leveraging BCI paradigms to enhance motor imagery ability
- Clinical validation and regulatory approval pathways for medical applications

5.2.7 Community and Infrastructure Development

Sustainable long-term impact requires:

- Establishment of open-source BCI software ecosystem with standardized interfaces and libraries
- Community contributions including new preprocessing pipelines, classifiers, and game integrations
- Standardized benchmarking datasets and evaluation metrics for comparative performance assessment
- Collaboration with disability advocacy organizations to ensure solutions address real user needs

The MindPlay project has established a foundation for practical brain-computer interface applications in gaming and accessibility. The methodologies developed herein provide a starting point for more sophisticated BCI systems addressing broader applications in healthcare, communication, and human-computer interaction research.

Bibliography

[1] J. R. Wolpaw, N. Birbaumer, D. J. McFarland, G. Pfurtscheller, and T. M. Vaughan, "Brain-computer interfaces for communication and control," *Clinical Neurophysiology*, vol. 113, no. 6, pp. 767–791, 2002.

[2] A. Delorme and S. Makeig, "Eeglab: an open source toolbox for analysis of brain topics from large-scale eeg studies," *Journal of Neuroscience Methods*, vol. 134, no. 1, pp. 9–21, 2004.

[3] G. R. Müller-Putz and R. Riedl, "Brain-computer interfaces for gaming and virtual reality," *Current Opinion in Behavioral Sciences*, vol. 29, pp. 34–38, 2019.

[4] B. Blankertz, R. Tomioka, S. Lemm, M. Kawanabe, and K.-R. Müller, "Optimizing spatial filters for robust eeg single-trial analysis," *IEEE Signal Processing Magazine*, vol. 25, no. 1, pp. 41–56, 2008.

[5] F. Lotte, L. Bougrain, and A. Cichocki, *Brain-Computer Interfaces: Handbook of Biofeedback and Advanced Neurofeedback*. Academic Press, 2018.

[6] A. Bashashati, M. Fatourechi, R. K. Ward, and G. E. Birch, "Real-time decoding of brain signals: a survey," *IEEE Transactions on Biomedical Engineering*, vol. 54, no. 12, pp. 1925–1935, 2007.

[7] C. A. Kothe and S. Makeig, "Lab streaming layer: a system for unified collection of measurement time series data in research experiments," *Software Impacts*, 2014.

[8] G. R. Müller-Putz, R. Scherer, and G. Pfurtscheller, "Surface-recorded brain signals for brain-computer interfaces," *Reviews in the Neurosciences*, vol. 19, no. 2-3, pp. 137–148, 2008.

[9] C. Neuper, R. Scherer, M. Reiter, and G. Pfurtscheller, "Motor imagery and action observation: cognitive tools for brain-computer interfaces," *Journal of Neuroscience Methods*, vol. 141, no. 2, pp. 198–211, 2005.

[10] K. K. Ang, Z. Y. Chin, C. Wang, and C. Guan, "Filter bank common spatial pattern in brain-computer interface," *IEEE Transactions on Biomedical Engineering*, vol. 55, no. 11, pp. 2766–2772, 2008.

[11] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936.

[12] R. T. Schirrmeister, J. T. Springenberg, L. D. Fiederer, M. Glasstetter, K. Eggensperger, M. Tangermann, F. Hutter, W. Burgard, and T. Ball, "Deep learning with convolutional neural networks for eeg decoding and visualization," *Human Brain Mapping*, vol. 38, no. 11, pp. 5391–5420, 2017.

[13] H. Higashi and T. Tanaka, "Physionet: physiologic signal archives for biomedical research," *IEEE Transactions on Biomedical Engineering*, vol. 60, no. 3, pp. 502–509, 2013.

[14] R. P. N. Rao and P. P. Mitra, "Eeg signal classification using wavelet feature extraction and neural networks," *Sensors*, vol. 20, no. 13, p. 3689, 2020.

Appendix A

Technical Diagrams and Supporting Material

A.1 System Design Diagrams

A.1.1 Use Case Diagram

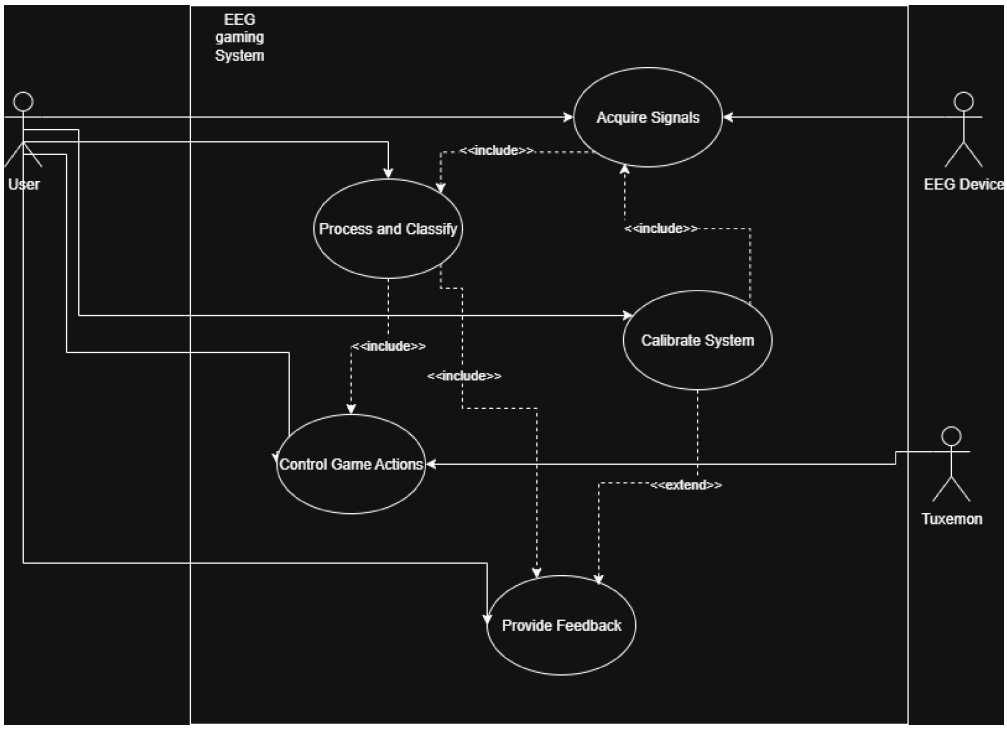


Figure A.1: Use Case Diagram for MindPlay - Showing interactions between users, system, and game environment

A.1.2 Domain Model

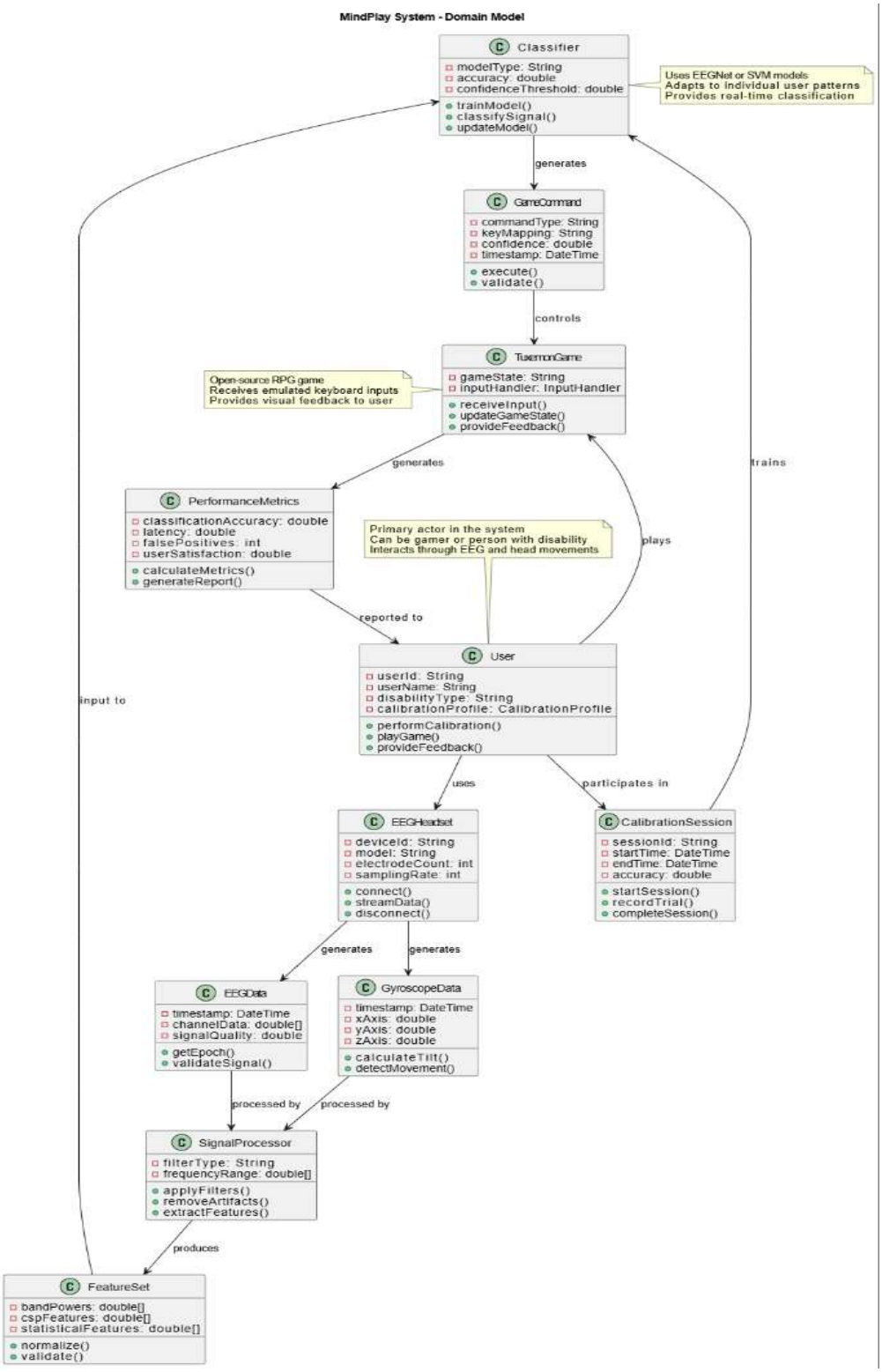


Figure A.2: Domain Model for MindPlay - Core entities and relationships

A.2 Process and Activity Diagrams

A.2.1 UML Activity Diagram

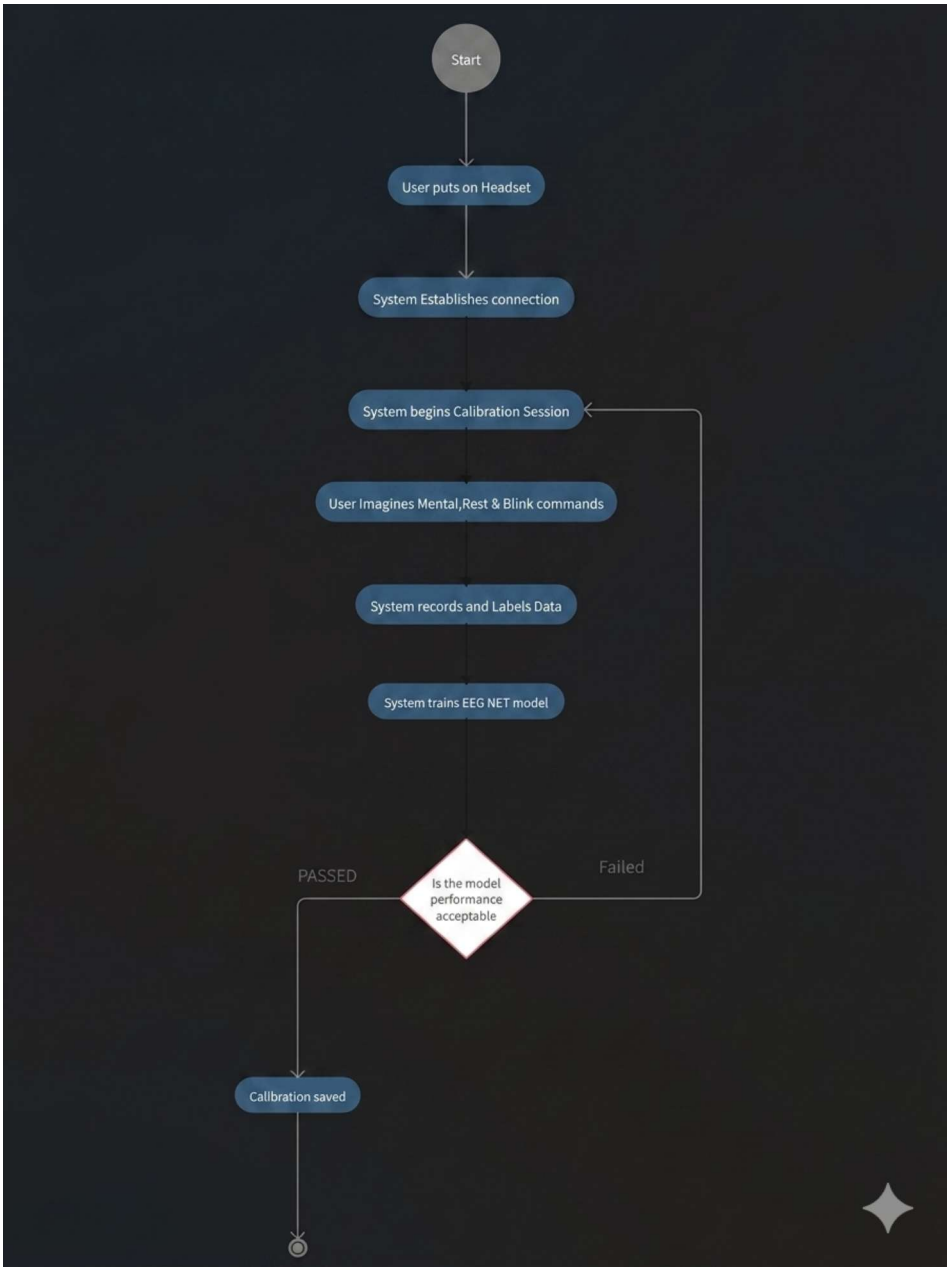


Figure A.3: UML Activity Diagram of the MindPlay EEG-Based Game Control Workflow showing calibration, signal acquisition, preprocessing, feature extraction, classification, and command execution

A.3 Data Flow Diagrams

A.3.1 System Level Data Flow

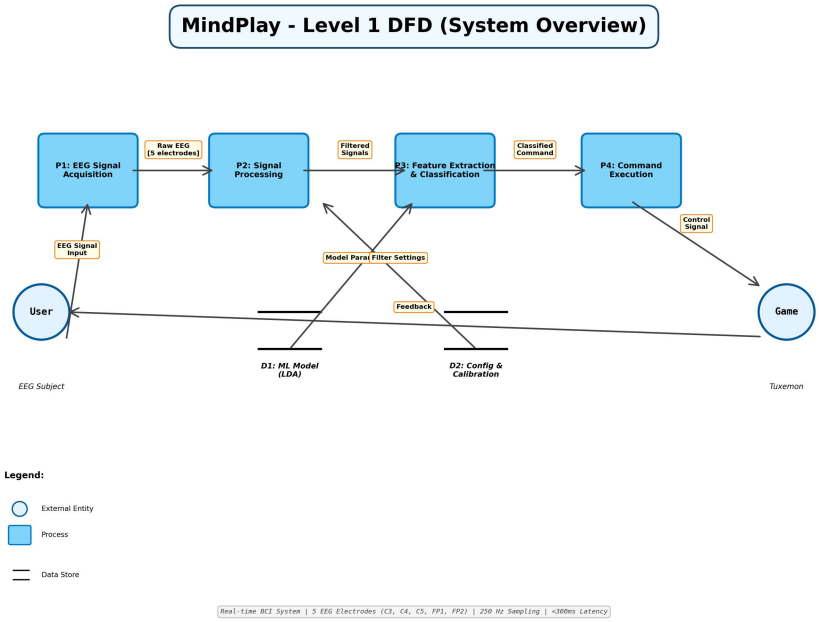


Figure A.4: Level 1 Data Flow Diagram for MindPlay - Overview of major data flows and system components

A.3.2 Detailed Component Data Flow

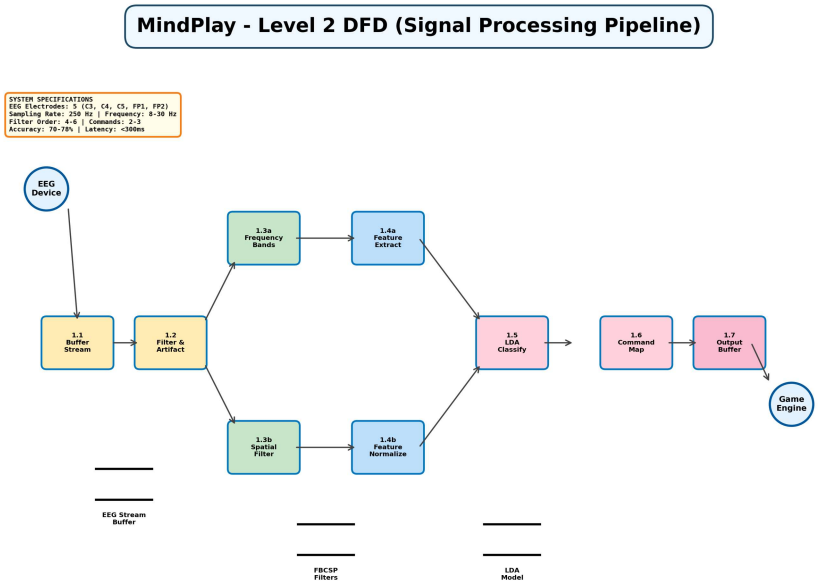


Figure A.5: Level 2 Data Flow Diagram for MindPlay - Detailed flows within core processing modules

A.4 Signal Processing Pipeline

A.4.1 FBCSP Processing Pipeline

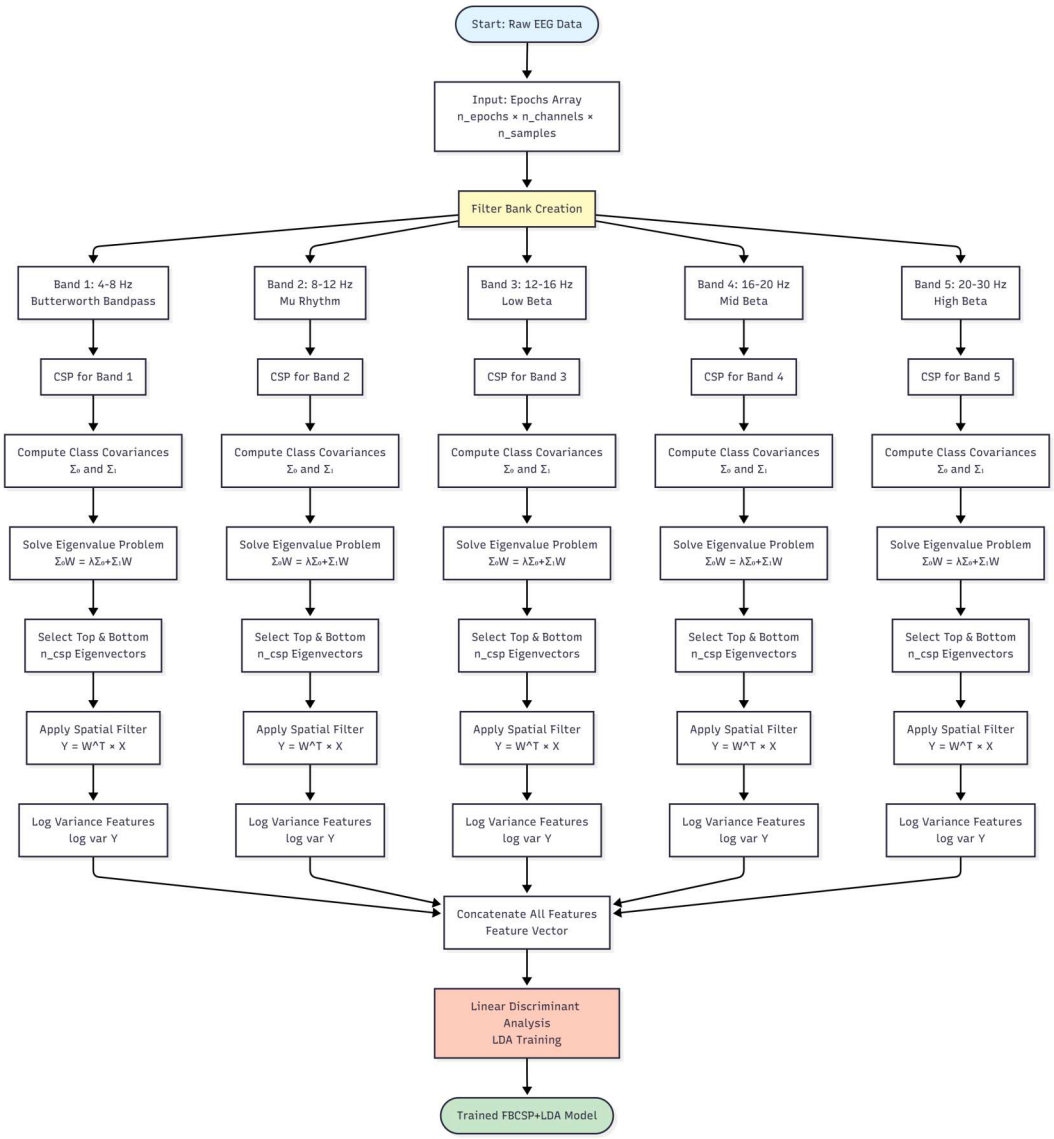
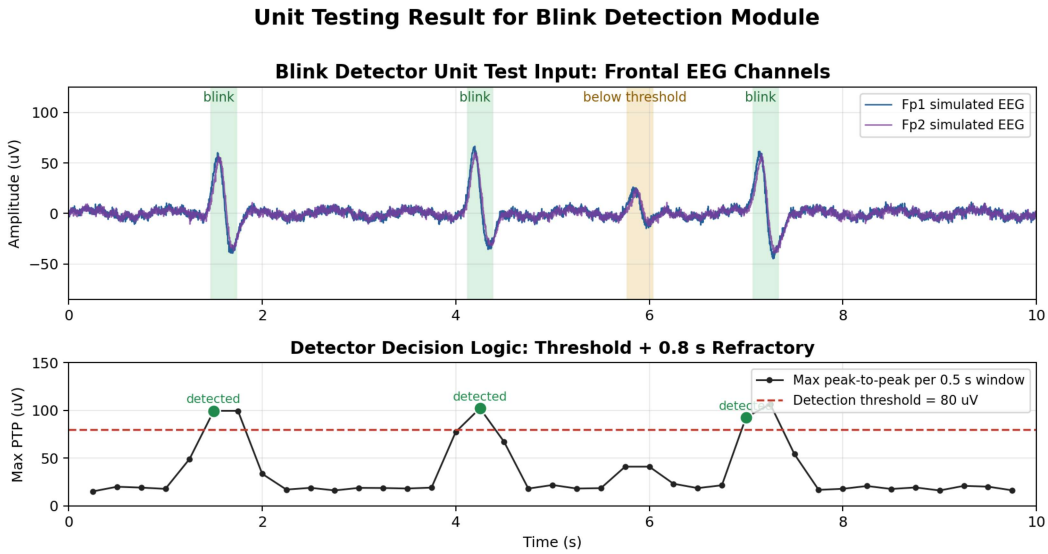


Figure A.6: Filter Bank Common Spatial Pattern (FBCSP) Signal Processing Pipeline - Showing frequency decomposition and spatial filtering stages

A.5 Testing and Validation Results

A.5.1 Unit Test Results



Test condition	Expected	Actual result	Status
3 intentional blink events > 80 uV	3 detections	3 detections	PASS
Subthreshold movement < 80 uV	No detection	Ignored	PASS
Refractory interval = 0.8 s	No duplicate triggers	3 accepted events	PASS

Test mirrors scripts/blink_detector.py: frontal picks Fp1/Fp2, 0.5 s window, peak-to-peak threshold, refractory suppression.

Figure A.7: Blink Detection Unit Test Results - Validation of eye blink detection algorithm performance

A.6 Implementation Resources

A.6.1 Software Dependencies

The MindPlay project relies on the following key open-source libraries and frameworks:

- **PyLSL:** Lab Streaming Layer Python bindings used for real-time EEG and IMU/gyroscope stream acquisition from the Smarting headset.
- **NumPy:** Fundamental numerical computing library used for EEG epoch buffers, matrix operations, covariance computation, and log-variance feature extraction.
- **SciPy:** Scientific computing library used for fourth-order Butterworth bandpass filtering and generalized eigenvalue decomposition during CSP filter computation.

- **Scikit-Learn:** Machine learning library used for Linear Discriminant Analysis (LDA), cross-validation, and evaluation metrics. CSP filters are implemented manually in the project code, not provided by scikit-learn.
- **Joblib:** Model persistence library used to save and load trained FBCSP+LDA models as `.joblib` files.
- **PyQt6:** Desktop GUI framework used for the launcher, training interface, real-time classifier controls, and module management.
- **pydirectinput:** Input automation library used to emit keyboard/game-control actions during real-time gameplay.
- **pynput:** Keyboard listener library used for hotkey handling and runtime control of detection modules.
- **Matplotlib:** Visualization library used for offline reporting, including accuracy plots, confusion matrices, and analysis figures.

A.6.2 Hardware Specifications

The physical hardware configuration utilized in the MindPlay system includes:

- **EEG Headset:** Smarting wireless EEG system configured for the final 3-electrode motor imagery montage (Cz, C3, C4) over central motor cortex and 2-electrode setup (Fp1,Fp2) for blink detection.
- **Sampling Rate:** 500 Hz across all EEG channels, providing 2 ms temporal resolution suitable for real-time motor imagery decoding
- **Built-in Gyroscope:** 3-axis motion sensor providing head movement data at synchronized timestamps via LSL framework
- **Processing Computer:** Standard desktop or laptop with minimum 8GB RAM and quad-core processor for real-time signal processing.
- **Operating System:** Windows 10/11 or Linux, with LSL network stack for hardware-software communication

A.6.3 Configuration Parameters

Key hyperparameters and settings used in system operation:

Table A.1: Critical System Configuration Parameters

Parameter	Value	Description
EEG Sampling Rate	500 Hz	Temporal resolution for signal acquisition
Filter Bank Bands	4-30 Hz	Frequency range for motor imagery decoding
CSP Filter Number	4	Spatial filters per frequency band
Window Length	4 seconds	Temporal window for feature extraction and classification decision
Window Step	0.5 seconds	Sliding window overlap for continuous operation
LDA Components	2	Binary classification (Rest vs. Motor Imagery)
Confidence Threshold	0.65	Minimum confidence for command execution
Debounce Duration	300 ms	Minimum interval between successive commands

A.7 Troubleshooting and Common Issues

A.7.1 Signal Quality Issues

Poor EEG signal quality can significantly degrade classification performance. Common causes and mitigation strategies:

1. **Electrode Contact:** Ensure proper scalp contact with conductive gel. Verify impedance readings below 10 kΩ.
2. **Electromagnetic Interference:** Minimize distance from electrical appliances. Use shielded cables and grounded USB connections.
3. **User Artifacts:** Minimize eye movements and muscle tension during calibration. Practice relaxation techniques.
4. **Sampling Synchronization:** Verify LSL stream timestamps are synchronized. Check for network latency or dropped packets.

A.7.2 Classification Performance Degradation

When accuracy falls below expected levels:

1. **Recalibration:** Perform new calibration session with current user to adapt model to signal variations.
2. **Parameter Tuning:** Adjust confidence threshold or window overlap based on session-specific characteristics.

3. **Model Retraining:** If cross-session accuracy persists, retrain FBCSP+LDA on fresh data.
4. **Electrode Verification:** Check all 5 electrodes for proper contact and impedance.

A.8 Future Enhancement Opportunities

The MindPlay platform provides a solid foundation for numerous future extensions and enhancements:

A.8.1 Hardware Upgrades

- Integration with high-density electrode systems (64+ channels) for improved signal quality and classification accuracy
- Wireless battery-powered headsets for untethered operation enabling broader mobility during gaming
- Hybrid sensing combining EEG with functional near-infrared spectroscopy (fNIRS) for multimodal brain activity monitoring
- Wearable form factors enabling long-duration BCI usage in natural environments

A.8.2 Software Enhancements

- Deep learning architectures (CNNs, RNNs) for end-to-end EEG decoding without manual feature engineering
- Domain adaptation techniques enabling zero-calibration transfer across users and sessions
- Adaptive threshold mechanisms automatically adjusting confidence thresholds based on performance drift
- Real-time visualization dashboards providing users with immediate feedback on signal quality and performance metrics

A.8.3 Accessibility and User Experience

- Integration with assistive technology frameworks for broader accessibility ecosystem support

- Customizable user interfaces enabling personalization for diverse user preferences and abilities
- Longitudinal learning studies quantifying skill development and neuroplasticity effects with extended BCI usage
- Subjective usability assessments capturing user satisfaction and acceptance metrics